DSA5208: Scalable Distributed Computing for Data Science

Lecture 6: Machine Learning with MLlib and Cloud Computing

Zhenning Cai

Department of Mathematics, National University of Singapore

AY2026/2027, Semester 1

1 Introduction to MLlib
2 Supervised machine learning with MLlib
3 Distributed algorithms for machine learning
4 Cloud computing and Google cloud platform
5 Apache Spark with Managed Service on Google Cloud Platform
6 Summary

What is MLlib?

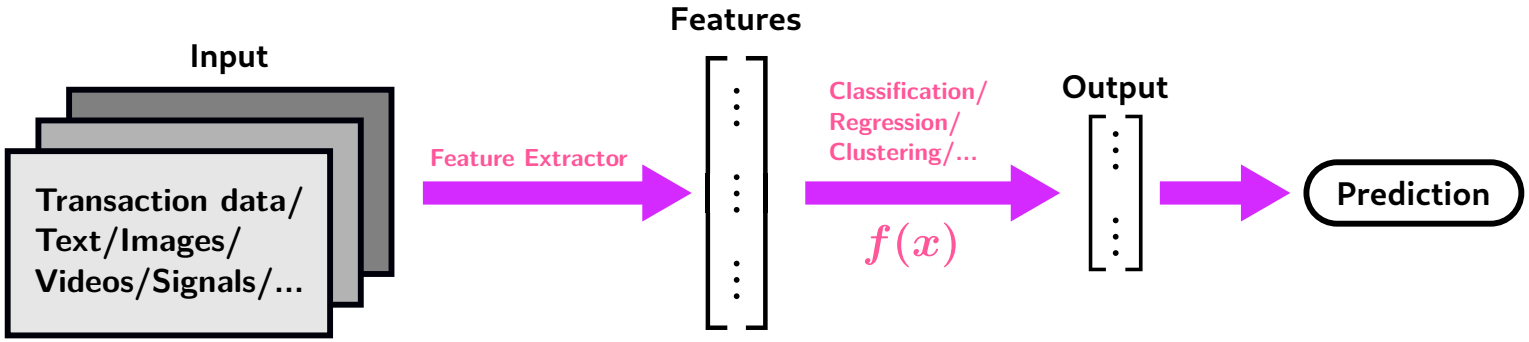
MLlib is Apache Spark's Machine Learning Library, designed for simplicity, scalability, and easy integration with other tools.

What can MLlib do?

- Classification (e.g. decision tree)
- Regression (e.g. linear regression)
- Collaborative filtering (e.g. alternating least squares)
- Clustering (e.g. k-means)
- Dimensionality reduction (e.g. principal component analysis)

A brief introduction to machines learning

A typical machine learning model:

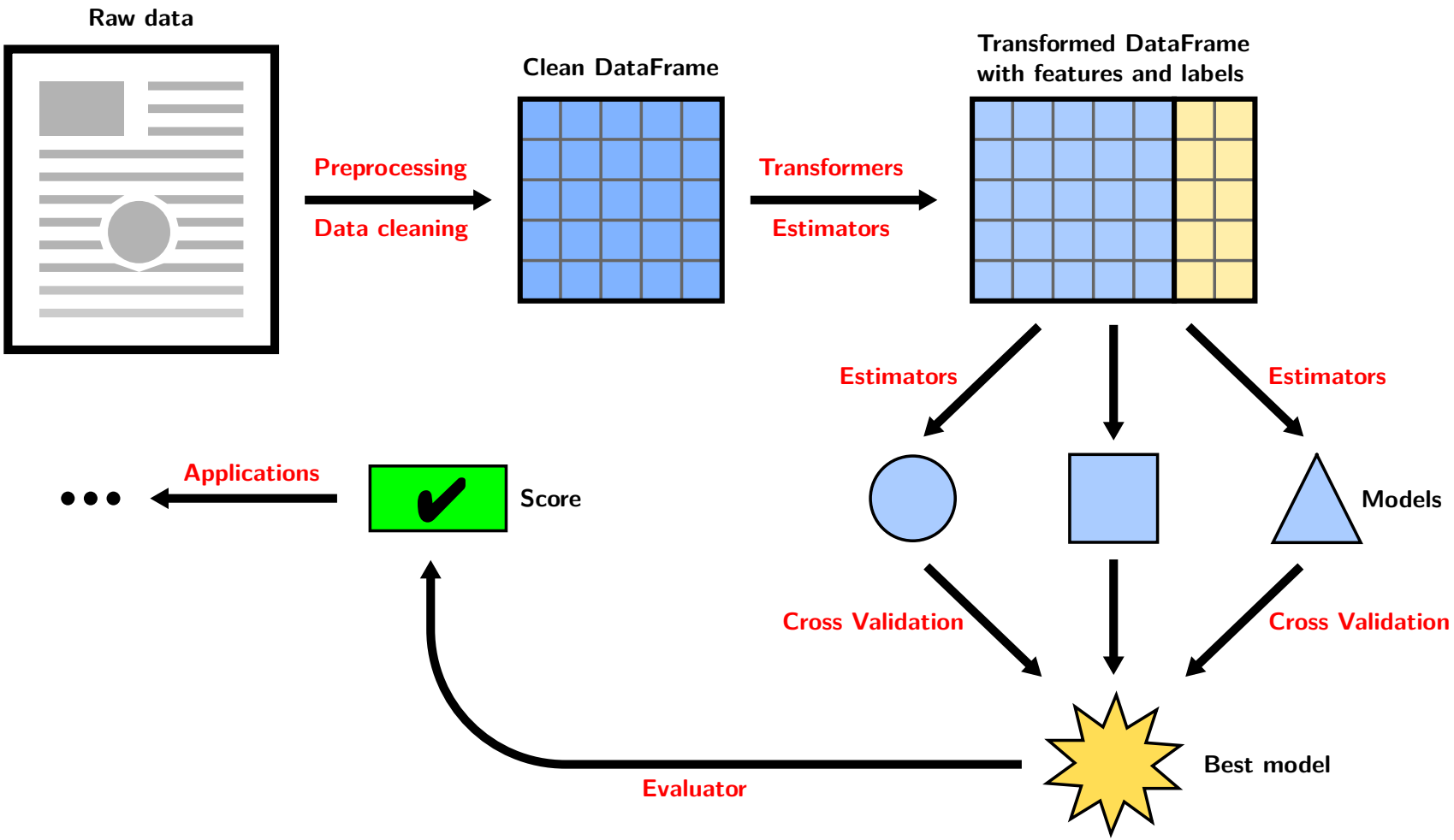


Machine learning learns the function $f(x)$ by fitting the training data according to certain rules.

Examples:

- Supervised learning: Given $f(x_i) = y_i$, $i = 1, \cdots, N$, find $f(x)$.
- Clustering: Given x_i, $i = 1, \cdots, N$, find $f(x)$ such that $f(x_i) = f(x_j)$ if x_i and x_j are "close" to each other.

How MLlib works



1 Introduction to MLlib

2 Supervised machine learning with MLlib

3 Distributed algorithms for machine learning

4 Cloud computing and Google cloud platform

5 Apache Spark with Managed Service on Google Cloud Platform

6 Summary

1 Introduction to MLlib

2 Supervised machine learning with MLlib
- A basic example of classification
- Pipelines and hyperparameter tuning
- Artificial neural network
- Regression

3 Distributed algorithms for machine learning

4 Cloud computing and Google cloud platform

5 Apache Spark with Managed Service on Google Cloud Platform

6 Summary

Iris flower dataset

Three species of Iris flowers:

Iris setosa



Iris versicolor



Iris virginica



Figures from `https://en.wikipedia.org/wiki/Iris_flower_data_set`

Iris flower dataset

Three species of Iris flowers:

Iris setosa



Iris versicolor



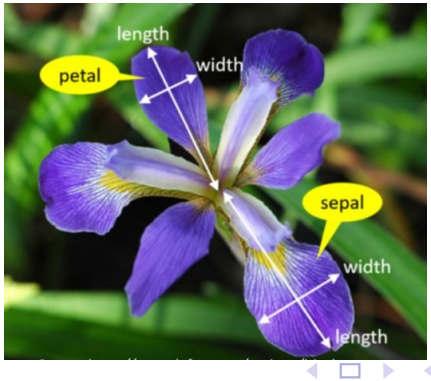
Iris virginica



Figures from `https://en.wikipedia.org/wiki/Iris_flower_data_set`

The dataset includes four features and one column of the species name.
The features are:

- Sepal length
- Sepal width
- Petal length
- Petal width



Iris flower dataset

Dataset:[1]

Sepal length (cm)	Sepal width (cm)	Petal length (cm)	Petal width (cm)	Species
5.1	3.5	1.4	0.2	*setosa*
4.9	3.0	1.4	0.2	*setosa*
⋮	⋮	⋮	⋮	⋮
5.0	3.3	1.4	0.2	*setosa*
7.0	3.2	4.7	1.4	*versicolor*
6.4	3.2	4.5	1.5	*versicolor*
⋮	⋮	⋮	⋮	⋮
5.7	2.8	4.1	1.3	*versicolor*
6.3	3.3	6.0	2.5	*virginica*
5.8	2.7	5.1	1.9	*virginica*
⋮	⋮	⋮	⋮	⋮
5.9	3.0	5.1	1.8	*virginica*

Task: Determine the species according to the sepal length/width and the petal length/width.

[1] R.A. Fisher, *The use of multiple measurements in taxonomic problems*, Annals of Eugenics. 7(2): 179–188, 1936.

Feature extraction

- **Read the dataset:**

```
>>> iris = spark.read.option("header", True).\
...                   option("inferSchema", True).csv("iris.csv")
```

- **Apply the transformer:** Use `VectorAssembler` to combine the four feature columns into one:

```
>>> from pyspark.ml.feature import VectorAssembler
>>> va = VectorAssembler(outputCol="features").setInputCols(iris.columns[0:4])
>>> preparedDF = va.transform(iris)
```

- **Apply the estimator:** Use `StringIndexer` to convert the species names to numerals:

```
>>> from pyspark.ml.feature import StringIndexer
>>> str_id = StringIndexer(outputCol="label").setInputCol("variety")
>>> preparedDF = str_id.fit(preparedDF).transform(preparedDF)
```

Transformers and estimators for preprocessing

- **Transformer:** Transformers convert one DataFrame into another, offen by appending one or more columns. A transformer always has a method `transform()`, whose first argument is always the DataFrame.

  Example:

  - Vector assembler: `va`
  - `transform()`: Add a column by assembling other columns.

Transformers and estimators for preprocessing

- **Transformer:** Transformers convert one DataFrame into another, offen by appending one or more columns. A transformer always has a method `transform()`, whose first argument is always the DataFrame.

  Example:

  - Vector assembler: `va`
  - `transform()`: Add a column by assembling other columns.

- **Estimator:** Estimators accept a DataFrame and produce a transformer. An estimator has a method `fit()`, which configures the output transformer by scanning the DataFrame.

  Example:

  - String indexer: `str_id`
  - `fit()`: Scan the column `variety` to check how many different strings exist and how to map each string to a unique number.
  - Result: `str_id.fit(preparedDF)` is a transformer that can be used to transform the DataFrame.

Train the model

- Split the dataset into the training set and the test set:

```
>>> train, test = preparedDF.randomSplit([.7, .3])
```

Train the model

- Split the dataset into the training set and the test set:

```
>>> train, test = preparedDF.randomSplit([.7, .3])
```

- Apply the estimator LogisticRegression to fit the training data:

```
>>> from pyspark.ml.classification import LogisticRegression
>>> lr = LogisticRegression().setFeaturesCol("features").\
                              setLabelCol("label")
>>> fittedLR = lr.fit(train)
```

The model has been trained!

Train the model

- Split the dataset into the training set and the test set:

```
>>> train, test = preparedDF.randomSplit([.7, .3])
```

- Apply the estimator LogisticRegression to fit the training data:

```
>>> from pyspark.ml.classification import LogisticRegression
>>> lr = LogisticRegression().setFeaturesCol("features").\
                              setLabelCol("label")
>>> fittedLR = lr.fit(train)
```

The model has been trained!

- Show the predictions of the model on the test dataset:

```
>>> fittedLR.transform(test).select("label", "prediction").show()
```

Train the model

- Split the dataset into the training set and the test set:

```
>>> train, test = preparedDF.randomSplit([.7, .3])
```

- Apply the estimator LogisticRegression to fit the training data:

```
>>> from pyspark.ml.classification import LogisticRegression
>>> lr = LogisticRegression().setFeaturesCol("features").\
                              setLabelCol("label")
>>> fittedLR = lr.fit(train)
```

The model has been trained!

- Show the predictions of the model on the test dataset:

```
>>> fittedLR.transform(test).select("label", "prediction").show()
```

- Use an evaluator to check the accuracy:

```
>>> from pyspark.ml.evaluation import MulticlassClassificationEvaluator
>>> evaluator = MulticlassClassificationEvaluator().\
...     setPredictionCol("prediction").setLabelCol("label")
>>> evaluator.evaluate(fittedLR.transform(train))
>>> evaluator.evaluate(fittedLR.transform(test))
```

Models and evaluators

- **Model:** A transformer representing the function $f(x)$.
  - Usually obtained by the `fit()` method of an estimator (`lr` in the example above).
  - The method `transform()` of a model computes the function $f(x)$ for all values in the input column, and then generates the output value in a new column.
  - Multiple new columns may be generated by some models.

  Example: The model `fittedLR` in the example above is a logistic regression model. Its `transform()` method adds two columns with default name `prediction` and `probability` to the DataFrame.

Models and evaluators

- **Model:** A transformer representing the function $f(x)$.
  - Usually obtained by the `fit()` method of an estimator (`lr` in the example above).
  - The method `transform()` of a model computes the function $f(x)$ for all values in the input column, and then generates the output value in a new column.
  - Multiple new columns may be generated by some models.

  Example: The model `fittedLR` in the example above is a logistic regression model. Its `transform()` method adds two columns with default name `prediction` and `probability` to the DataFrame.
- **Evaluator:** An evaluator of a given model.
  - Has a method `evaluate()` taking the dataset transformed by a model as its parameter.
  - Produces a score by comparing the true and predicted labels.

  Example: The variable `evaluator` in the example above is an evaluator for multiclass classifications.

1 Introduction to MLlib

2 Supervised machine learning with MLlib
- A basic example of classification
- Pipelines and hyperparameter tuning
- Artificial neural network
- Regression

3 Distributed algorithms for machine learning

4 Cloud computing and Google cloud platform

5 Apache Spark with Managed Service on Google Cloud Platform

6 Summary

Tuning

Each transformer or estimator are often accompannied with several parameters that may change its output.

- Parameters of `str_id`:

```
>>> print(str_id.explainParams())
```

- Parameters of `lr`:

```
>>> print(lr.explainParams())
```

- We can change the parameter of `lr` and test the performance again:

```
>>> lr.setRegParam(1)
>>> fittedLR = lr.fit(train)
>>> evaluator.evaluate(fittedLR.transform(train))
>>> evaluator.evaluate(fittedLR.transform(test))
```

Pipeline

Can we get better results? Let's add some other features such as the approximate areas (width $\times$ length) of the sepal and the petal:

```
>>> from pyspark.ml.feature import RFormula
>>> rf = RFormula(formula =
...     "variety ~ . + sepal_length:sepal_width + petal_length:petal_width")
>>> preparedDF = rf.fit(iris).transform(iris)
>>> train, test = preparedDF.randomSplit([.7, .3])
>>> lr = LogisticRegression()
>>> fittedLR = lr.fit(train)
>>> evaluator.evaluate(fittedLR.transform(train))
>>> evaluator.evaluate(fittedLR.transform(test))
```

Two estimators are applied in the code above. We can combine them into a pipeline:

```
>>> from pyspark.ml.pipeline import Pipeline
>>> pipeline = Pipeline(stages = [rf, lr])
>>> train, test = iris.randomSplit([.7, .3])
>>> model = pipeline.fit(train)
>>> evaluator.evaluate(model.transform(train))
>>> evaluator.evaluate(model.transform(test))
```

How a pipeline works

A Pipeline is specified as a sequence of stages, and each stage is either a transformer or an estimator.

How a pipeline works

A Pipeline is specified as a sequence of stages, and each stage is either a transformer or an estimator.

- A pipeline is an estimator that has a `fit()` method, which produces a pipeline model.

How a pipeline works

A Pipeline is specified as a sequence of stages, and each stage is either a transformer or an estimator.

- A pipeline is an estimator that has a `fit()` method, which produces a pipeline model.
- When calling `fit()` on a DataFrame, all estimator stages are fit by the output of the previous stage.

How a pipeline works

A Pipeline is specified as a sequence of stages, and each stage is either a transformer or an estimator.

- A pipeline is an estimator that has a `fit()` method, which produces a pipeline model.
- When calling `fit()` on a DataFrame, all estimator stages are fit by the output of the previous stage.
  - If a stage is a transformer, its output is the result of the `transform()` method.

How a pipeline works

A Pipeline is specified as a sequence of stages, and each stage is either a transformer or an estimator.

- A pipeline is an estimator that has a `fit()` method, which produces a pipeline model.
- When calling `fit()` on a DataFrame, all estimator stages are fit by the output of the previous stage.
  - If a stage is a transformer, its output is the result of the `transform()` method.
  - If a stage is an estimator, its output is the result of the `transform()` method of the fitted model.

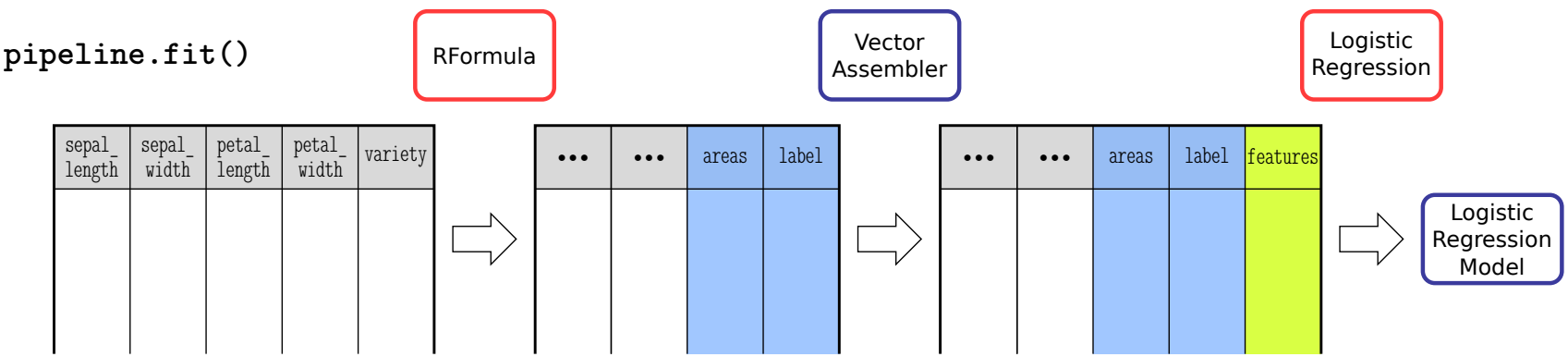
How a pipeline works

A Pipeline is specified as a sequence of stages, and each stage is either a transformer or an estimator.

- A pipeline is an estimator that has a `fit()` method, which produces a pipeline model.
- When calling `fit()` on a DataFrame, all estimator stages are fit by the output of the previous stage.
  - If a stage is a transformer, its output is the result of the `transform()` method.
  - If a stage is an estimator, its output is the result of the `transform()` method of the fitted model.
- A pipeline model is a transformer. When calling its `transform()` method on a DataFrame, all stages are run in order. For estimator stages, the model trained in the `fit()` method is run.

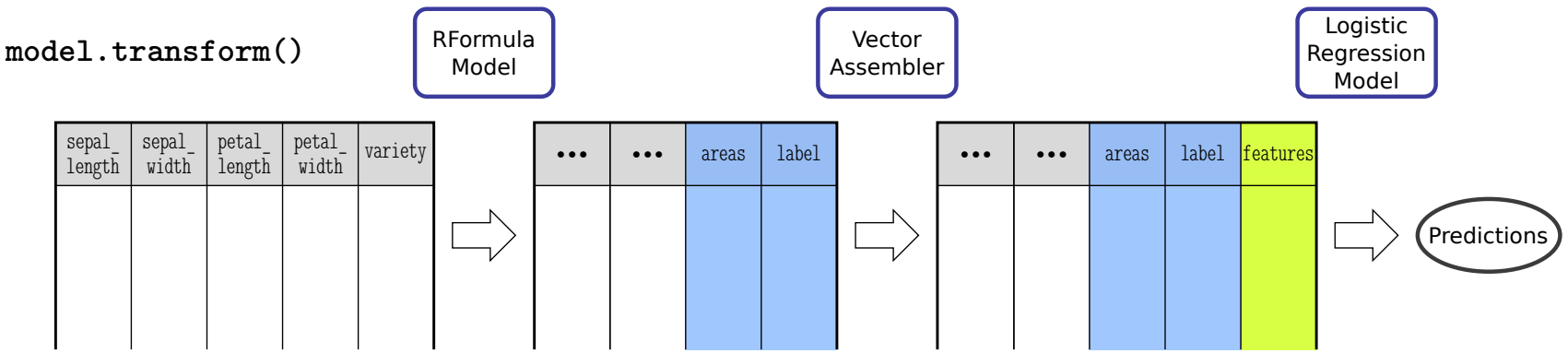
Example

```
>>> rf = RFormula().setFeaturesCol("areas").\
...     setFormula("variety ~ sepal_length:sepal_width + petal_length:petal_width")\
...     # First stage: estimator
>>> va = VectorAssembler(outputCol="features").\
...     setInputCols(iris.columns[0:4] + ["areas"]) # Second stage: transformer
>>> lr = LogisticRegression().setFeaturesCol("features").\
...     setLabelCol("label") # Third stage: estimator
>>> pipeline = Pipeline(stages = [rf, va, lr]) # Construct the pipeline
>>> model = pipeline.fit(train) # Fit training data to produce an ML model
```



Example

```
>>> pred = model.transform(iris)
>>> pred.sample(fraction = 0.1).select("label", "prediction").show()
```



Build a parameter grid

Consider the following pipeline:

```
>>> rf = RFormula(formula = "")
>>> lr = LogisticRegression()
>>> pipeline = Pipeline(stages = [rf, lr])
```

Both estimators `rf` and `lr` have hyperparameters that can be tuned:

- `rf.formula`:
  1. `rf1 = "variety ~ ."`
  2. `rf2 = "variety ~ . ^ 2"`
  3. `rf3 = "variety ~ . + sepal_length:sepal_width + petal_length:petal_width"`
- `lr.regParam`:
  1. 0.0
  2. 0.1
  3. 0.2

But what is the optimal combination?

Build a parameter grid

To find the optimal hyperparameters, we need to validate models on the following parameter grid:

		lr.regParam		
		0.0	0.1	0.2
rf.formula	rf2	Pipeline (0,0)	Pipeline (0,1)	Pipeline (0,2)
	rf1	Pipeline (1,0)	Pipeline (1,1)	Pipeline (1,2)
	rf0	Pipeline (2,0)	Pipeline (2,1)	Pipeline (2,2)

Build a parameter grid

To find the optimal hyperparameters, we need to validate models on the following parameter grid:

		lr.regParam		
		0.0	0.1	0.2
rf.formula	rf2	Pipeline (0,0)	Pipeline (0,1)	Pipeline (0,2)
	rf1	Pipeline (1,0)	Pipeline (1,1)	Pipeline (1,2)
	rf0	Pipeline (2,0)	Pipeline (2,1)	Pipeline (2,2)

This grid can be built by using `ParamGridBuilder`:

```
>>> from pyspark.ml.tuning import ParamGridBuilder
>>> rf0 = "variety ~ ."
>>> rf1 = "variety ~ . ^ 2"
>>> rf2 = "variety ~ . + sepal_length:sepal_width + petal_length:petal_width"
>>> params = ParamGridBuilder().\
...         addGrid(rf.formula, [rf0, rf1, rf2]).\
...         addGrid(lr.regParam, [0, 0.1, 0.2]).\
...         build()
```

Validate the models

MLlib provides automatic model tuning that can fit many models using a grid of parameters and pick the best model. One of such model selection tools is `CrossValidator`:

```
>>> from pyspark.ml.tuning import CrossValidator
>>> cv = CrossValidator().\
...         setNumFolds(4).\
...         setEstimatorParamMaps(params).\
...         setEstimator(pipeline).\
...         setEvaluator(evaluator)
>>> model = cv.fit(train) # Fit 9 models and pick the best one
>>> evaluator.evaluate(model.transform(iris))
```

- A `CrossValidator` is an estimator whose `fit()` method fits all models with hyperparameters in the parameter grid and picks the best model based on cross validation.
- The result of the `fit()` method is a transformer with the best parameters.

Cross validation

How does a `CrossValidator` evaluate a model only based on the training dataset?

Cross validation

How does a `CrossValidator` evaluate a model only based on the training dataset?

4-fold cross validation:

- Divide the dataset into 4 parts.

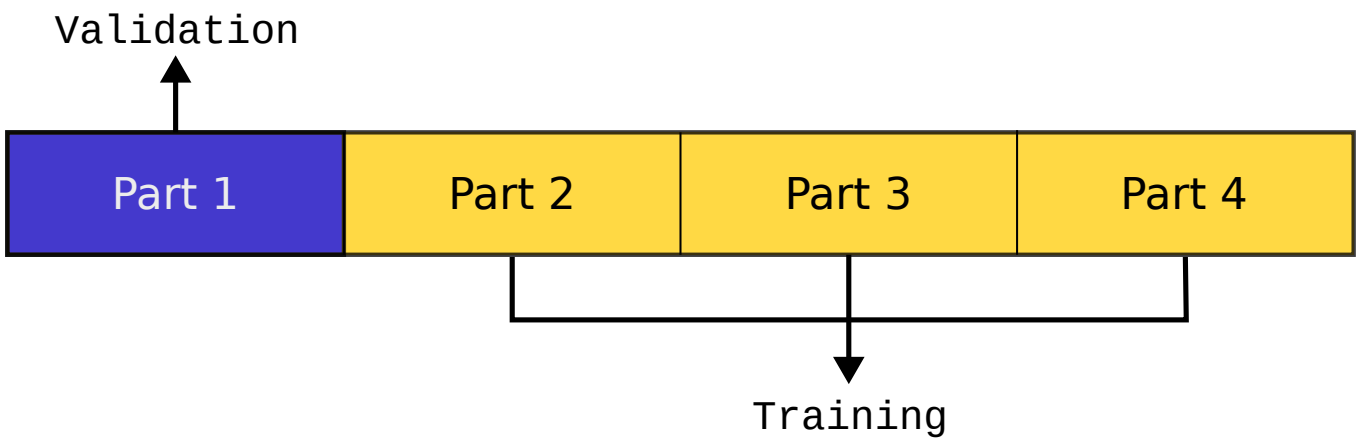


Cross validation

How does a `CrossValidator` evaluate a model only based on the training dataset?

4-fold cross validation:

- Divide the dataset into 4 parts.
- Train on three parts and validate on the remaining part.

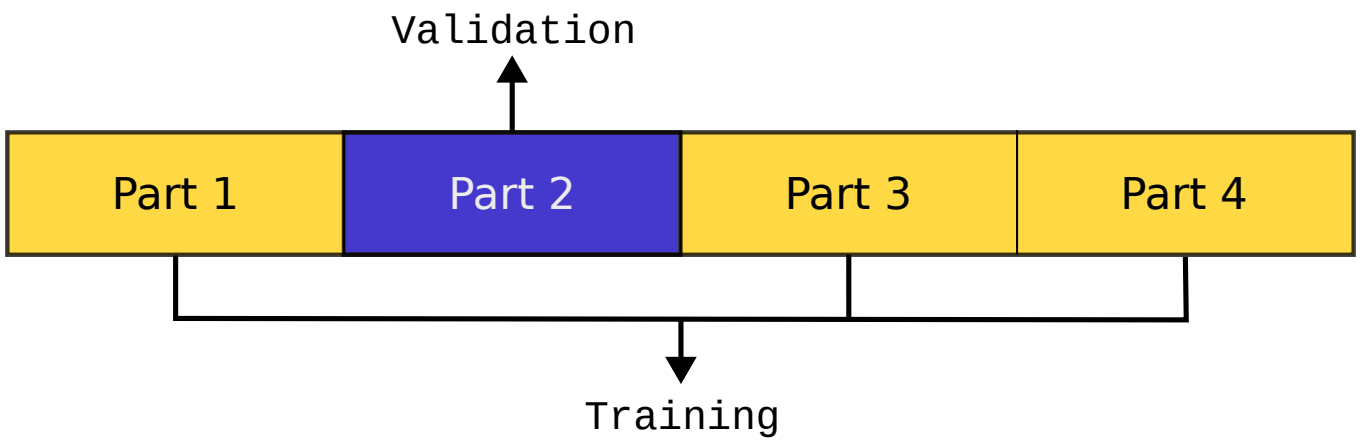


Cross validation

How does a `CrossValidator` evaluate a model only based on the training dataset?

4-fold cross validation:

- Divide the dataset into 4 parts.
- Train on three parts and validate on the remaining part.

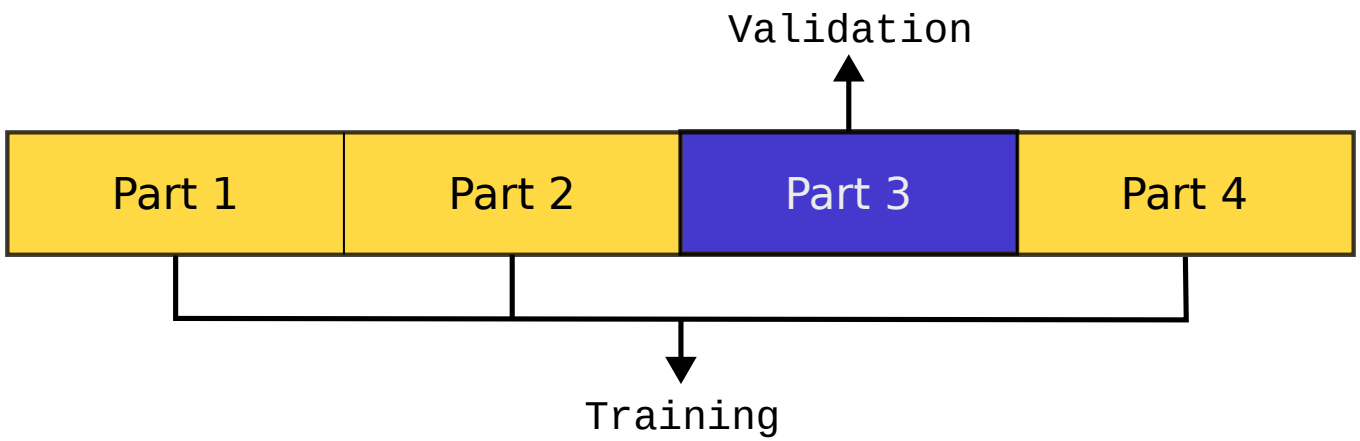


Cross validation

How does a `CrossValidator` evaluate a model only based on the training dataset?

4-fold cross validation:

- Divide the dataset into 4 parts.
- Train on three parts and validate on the remaining part.

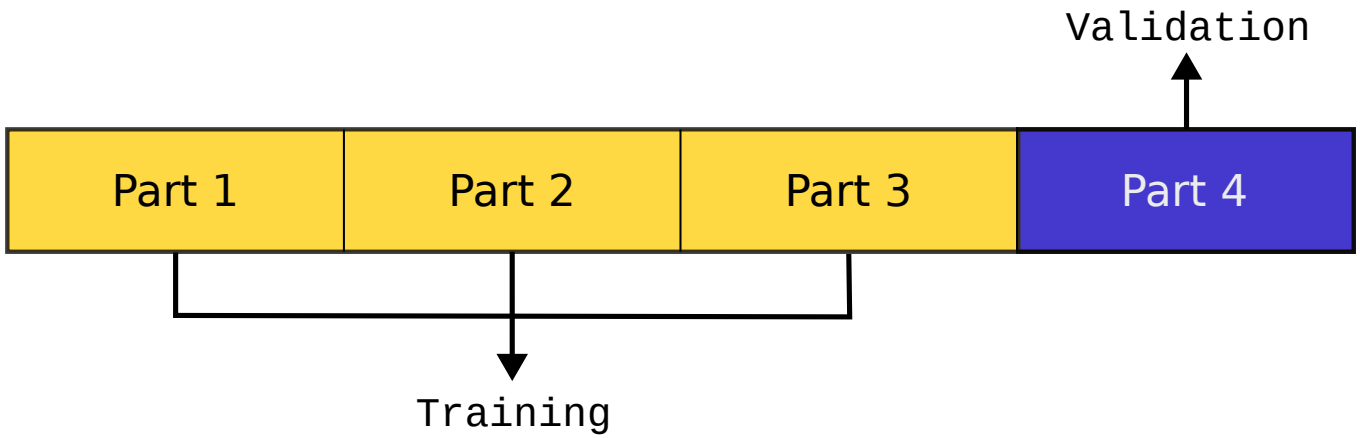


Cross validation

How does a `CrossValidator` evaluate a model only based on the training dataset?

4-fold cross validation:

- Divide the dataset into 4 parts.
- Train on three parts and validate on the remaining part.

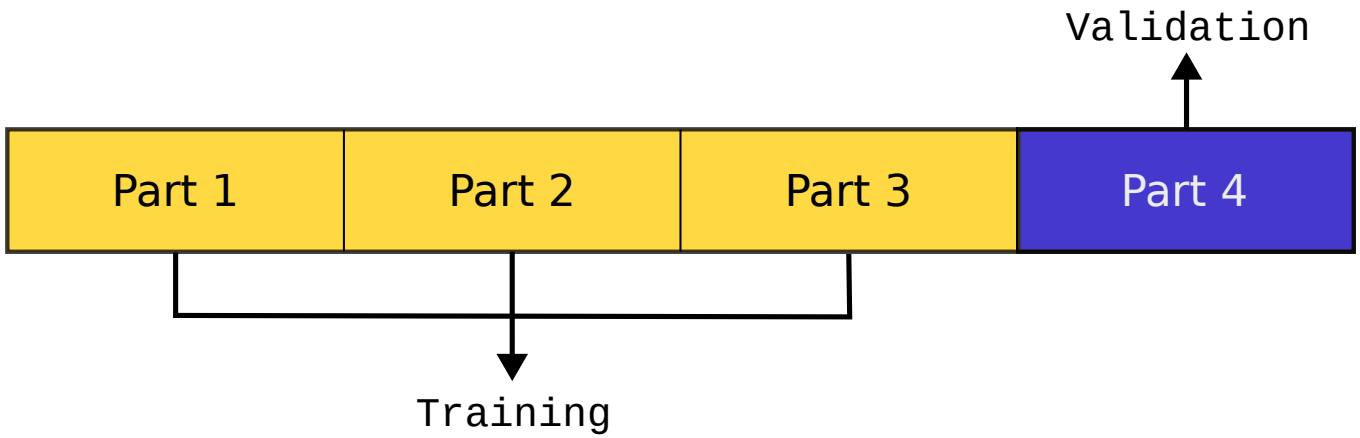


Cross validation

How does a `CrossValidator` evaluate a model only based on the training dataset?

4-fold cross validation:

- Divide the dataset into 4 parts.
- Train on three parts and validate on the remaining part.
- Compute the overall accuracy.

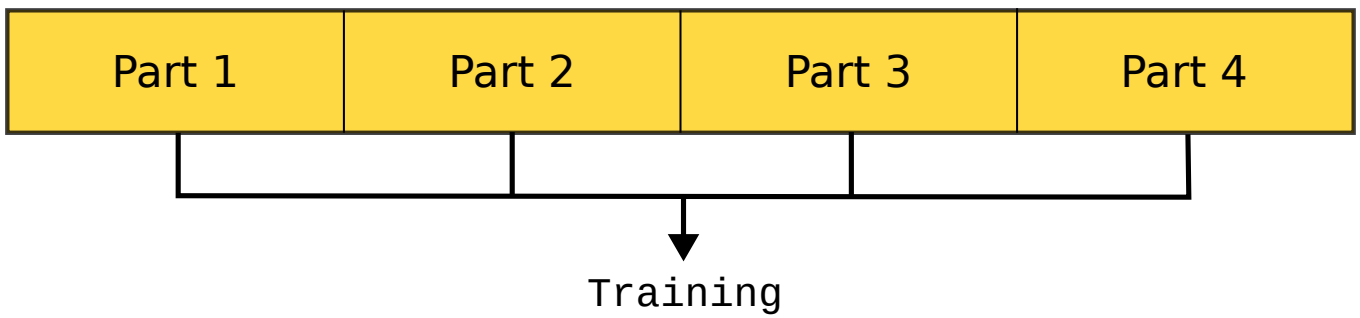


Cross validation

How does a `CrossValidator` evaluate a model only based on the training dataset?

4-fold cross validation:

- Divide the dataset into 4 parts.
- Train on three parts and validate on the remaining part.
- Compute the overall accuracy.
- For the best model, train on the entire dataset.



Train-Validation Split

Spark also offers `TrainValidationSplit` for hyper-parameter tuning.

- It splits the dataset into a training set and a validation set.
- It evaluates each model on the validation set only once.

```
>>> from pyspark.ml.tuning import TrainValidationSplit
>>> tvs = TrainValidationSplit().\
...         setTrainRatio(0.7).\
...         setEstimatorParamMaps(params).\
...         setEstimator(pipeline).\
...         setEvaluator(evaluator)
>>> model = tvs.fit(train)
>>> evaluator.evaluate(model.transform(iris))
```

1 Introduction to MLlib

2 Supervised machine learning with MLlib
- A basic example of classification
- Pipelines and hyperparameter tuning
- Artificial neural network
- Regression

3 Distributed algorithms for machine learning

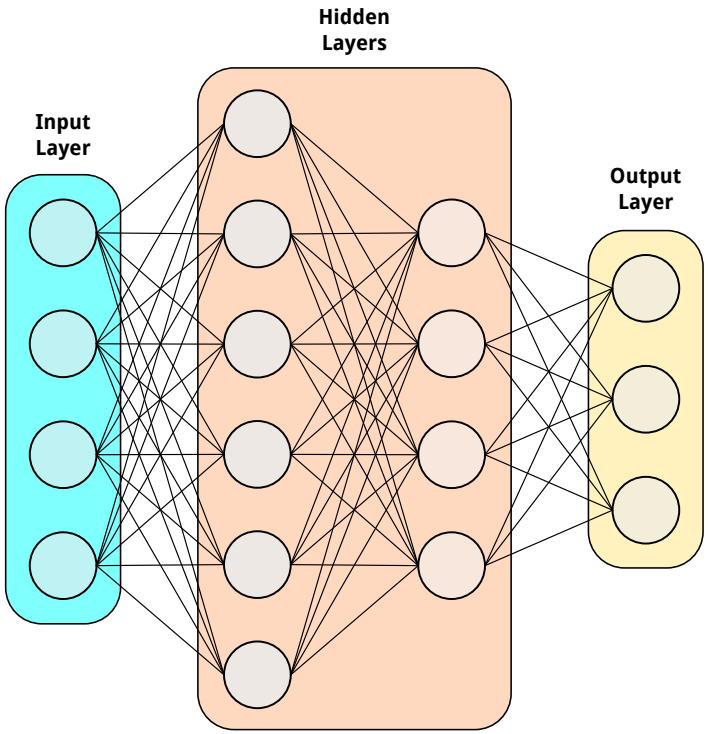
4 Cloud computing and Google cloud platform

5 Apache Spark with Managed Service on Google Cloud Platform

6 Summary

Multilayer perceptron classifier

Multilayer perceptron classifier (MPLC) is a classifier based on the artifical neural network:



$$
\begin{aligned}
f(x) = \sigma_K(&\\
&w_K^T \sigma_{K-1}(\\
&\quad \cdots\\
&\quad\quad w_2^T \sigma_1(\\
&\quad\quad\quad w_1^T x + b_1\\
&\quad\quad) + b_2\\
&\quad \cdots\\
&) + b_K\\
)&
\end{aligned}
$$

where $f(x)$ represents a $(K+1)$-layer artificial neural network, and $\sigma_1, \cdots, \sigma_N$ are activation functions.

Multilayer perceptron classifier

In MLlib's implementation,

$$\sigma_k(z) = \frac{1}{1 + \mathrm{e}^z} \qquad \text{(componentwise operation)}$$

if $k = 1, \cdots, K-1$, and

$$\sigma_K(z) = \frac{\mathrm{e}^z}{\|\mathrm{e}^z\|_1},$$

where $\|\cdot\|_1$ denotes the ℓ^1 norm of a vector.

Multilayer perceptron classifier

In MLlib's implementation,

$$\sigma_k(z) = \frac{1}{1+\mathrm{e}^z} \qquad \text{(componentwise operation)}$$

if $k = 1, \cdots, K-1$, and

$$\sigma_K(z) = \frac{\mathrm{e}^z}{\|\mathrm{e}^z\|_1},$$

where $\|\cdot\|_1$ denotes the ℓ^1 norm of a vector.

- The size of the input layer should equal the number of features.
- The size of the output layer should equal the number of classes.
- The ith component of $f(x)$ means the probability that x belongs to the ith class.

Example

The following code builds an artificial neural network with

- an input layer of size 4,
- two hidden layers of sizes 6 and 4,
- and an output layer of size 3.

```
>>> rf = RFormula(formula = "variety ~ .")
>>> from pyspark.ml.classification import MultilayerPerceptronClassifier
>>> mlpc = MultilayerPerceptronClassifier(layers=[4,6,4,3])
>>> pipeline = Pipeline(stages = [rf, mlpc])
>>> train, test = iris.randomSplit([.7, .3])
>>> model = pipeline.fit(train)
>>> evaluator.evaluate(model.transform(iris))
```

Example

The following code builds an artificial neural network with

- an input layer of size 4,
- two hidden layers of sizes 6 and 4,
- and an output layer of size 3.

```
>>> rf = RFormula(formula = "variety ~ .")
>>> from pyspark.ml.classification import MultilayerPerceptronClassifier
>>> mlpc = MultilayerPerceptronClassifier(layers=[4,6,4,3])
>>> pipeline = Pipeline(stages = [rf, mlpc])
>>> train, test = iris.randomSplit([.7, .3])
>>> model = pipeline.fit(train)
>>> evaluator.evaluate(model.transform(iris))
```

Can you apply cross validation to find a better architecture?

1 Introduction to MLlib

2 Supervised machine learning with MLlib
- A basic example of classification
- Pipelines and hyperparameter tuning
- Artificial neural network
- Regression

3 Distributed algorithms for machine learning

4 Cloud computing and Google cloud platform

5 Apache Spark with Managed Service on Google Cloud Platform

6 Summary

The California housing dataset

longitude	longitude	housing median age	total rooms	total bedrooms	...	median income	median housing value	ocean proximity
-122.23	37.88	41.0	880.0	129.0	...	8.3252	452600.0	NEAR BAY
-122.22	37.86	21.0	7099.0	1106.0	...	8.3014	358500.0	NEAR BAY
⋮	⋮	⋮	⋮	⋮		⋮	⋮	⋮
-121.32	39.43	18.0	1860.0	409.0	...	1.8672	84700.0	INLAND
-121.24	39.37	16.0	2785.0	616.0	...	2.3886	89400.0	INLAND

The California housing dataset

longitude	longitude	housing median age	total rooms	total bedrooms	...	median income	median housing value	ocean proximity
-122.23	37.88	41.0	880.0	129.0	...	8.3252	452600.0	NEAR BAY
-122.22	37.86	21.0	7099.0	1106.0	...	8.3014	358500.0	NEAR BAY
⋮	⋮	⋮	⋮	⋮		⋮	⋮	⋮
-121.32	39.43	18.0	1860.0	409.0	...	1.8672	84700.0	INLAND
-121.24	39.37	16.0	2785.0	616.0	...	2.3886	89400.0	INLAND

Question: Can we predict the median housing value based on other features?

The California housing dataset

longitude	longitude	housing median age	total rooms	total bedrooms	...	median income	median housing value	ocean proximity
-122.23	37.88	41.0	880.0	129.0	...	8.3252	452600.0	NEAR BAY
-122.22	37.86	21.0	7099.0	1106.0	...	8.3014	358500.0	NEAR BAY
⋮	⋮	⋮	⋮	⋮		⋮	⋮	⋮
-121.32	39.43	18.0	1860.0	409.0	...	1.8672	84700.0	INLAND
-121.24	39.37	16.0	2785.0	616.0	...	2.3886	89400.0	INLAND

Question: Can we predict the median housing value based on other features?

Compared with classification problems, regression problems predict continous values.

Ridge regression

Ridge regression: Given a dataset

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}, \qquad x_i \in \mathbb{R}^d, \quad y_i \in \mathbb{R},$$

find $w \in \mathbb{R}^d$ and $b \in \mathbb{R}$ that minimize the risk function

$$R(w, b) = \frac{1}{2N} \sum_{i=1}^{N} [(w^T x_i + b) - y_i]^2 + \frac{1}{2} w^T \Lambda w.$$

The trained model is $f(x) = w^T x + b$.

Ridge regression

Ridge regression: Given a dataset

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N, \qquad x_i \in \mathbb{R}^d, \quad y_i \in \mathbb{R},$$

find $w \in \mathbb{R}^d$ and $b \in \mathbb{R}$ that minimize the risk function

$$R(w, b) = \frac{1}{2N} \sum_{i=1}^{N} [(w^T x_i + b) - y_i]^2 + \frac{1}{2} w^T \Lambda w.$$

The trained model is $f(x) = w^T x + b$.

Regularizer: In ridge regression, the term $\frac{1}{2} w^T \Lambda w$ is called the regularizer, and Λ is a diagonal matrix whose diagonal entries are all nonnegative. The purpose of the regularizer is to encourage simple models and avoid overfitting.

Example

```
>>> housing = spark.read.option("header", True).\
...                       option("inferSchema", True).csv("housing.csv")
>>> housing = housing.na.drop(how="any") # Remove row with missing values
>>> from pyspark.ml.feature import RFormula
>>> rf = RFormula(formula = "median_house_value ~ .")
>>> from pyspark.ml.regression import LinearRegression
>>> lr = LinearRegression() # Estimator for linear regression
>>> from pyspark.ml.pipeline import Pipeline
>>> pipeline = Pipeline(stages = [rf, lr])
>>> from pyspark.ml.evaluation import RegressionEvaluator
>>> evaluator = RegressionEvaluator() # Evaluator for regression problems
>>> from pyspark.ml.tuning import ParamGridBuilder, CrossValidator
>>> params = ParamGridBuilder().addGrid(lr.regParam, [0, 0.1, 0.2]).\
...     build() # lr.regParam is the hyperparameter in the regularizer
>>> cv = CrossValidator().setNumFolds(4).setEstimatorParamMaps(params).\
>>>                       setEstimator(pipeline).setEvaluator(evaluator)
>>> train, test = housing.randomSplit([.7, .3])
>>> model = cv.fit(train)
>>> evaluator.evaluate(model.transform(train)) # Compute RMSE on the training set
>>> evaluator.evaluate(model.transform(test)) # Compute RMSE on the test set
```

Can you improve the result?

1 Introduction to MLlib

2 Supervised machine learning with MLlib

3 Distributed algorithms for machine learning

4 Cloud computing and Google cloud platform

5 Apache Spark with Managed Service on Google Cloud Platform

6 Summary

1 Introduction to MLlib

2 Supervised machine learning with MLlib

3 Distributed algorithms for machine learning
- Distributed algorithm for linear regression
- Distributed algorithm for alternating least squares

4 Cloud computing and Google cloud platform

5 Apache Spark with Managed Service on Google Cloud Platform

6 Summary

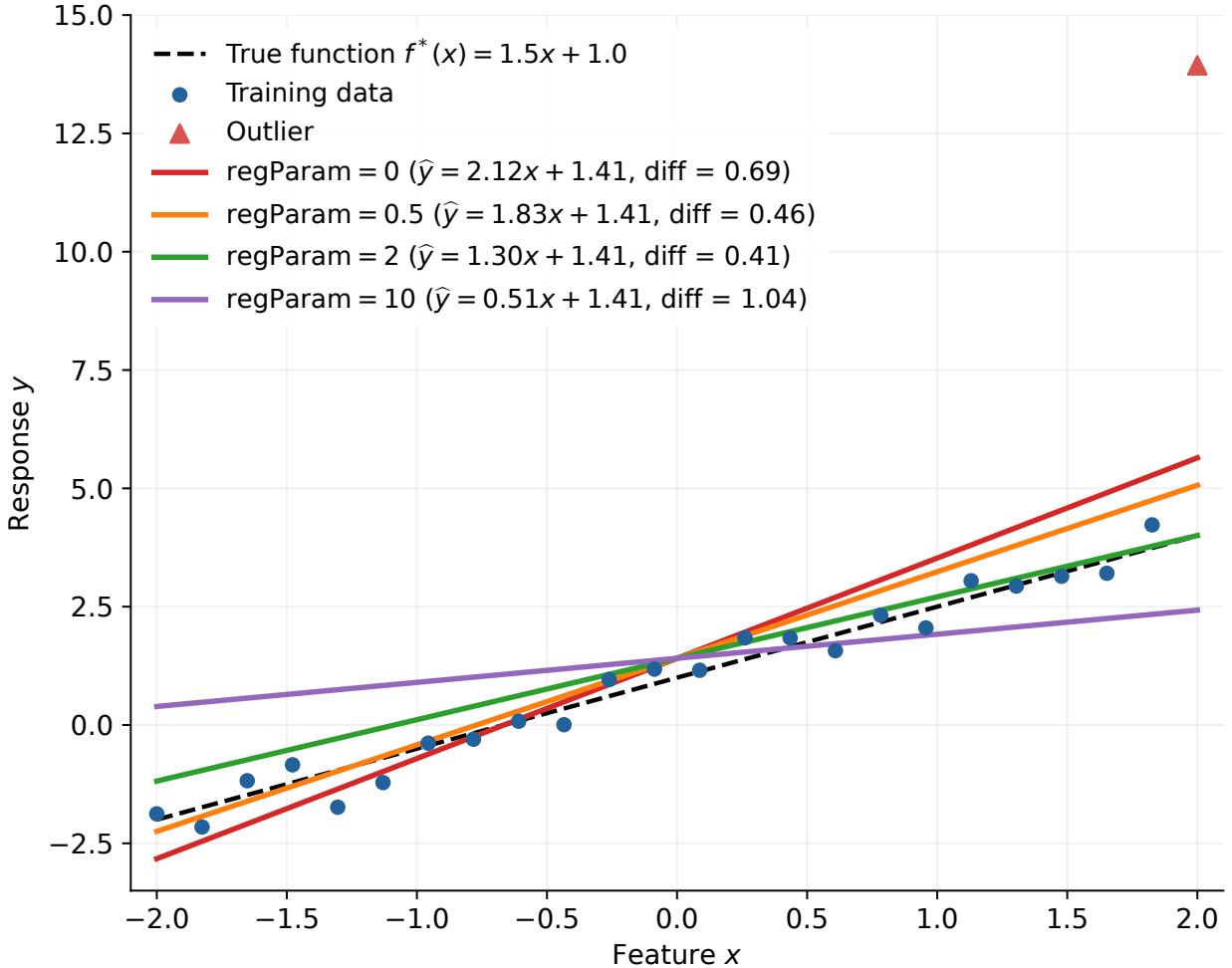
Ridge regression

Problem:

$$\min_{w,b} R(w,b) = \frac{1}{2\sum_{i=1}^{N} a_i} \sum_{i=1}^{N} a_i (w^T x_i + b - y_i)^2 + \frac{1}{2} w^T \Lambda w.$$

- Estimator in Spark: `LinearRegression`.
- Spark sets Λ using `regParam` and the data scales.
- Setting `fitIntercept=True` (default) enables the intercept b.
- a_i are observation weights; Spark's default is $a_i = 1$.

Effect of the regularization parameter



Equations for w and b

Assume equal observation weights $a_i = 1$. Define the sample means

$$\bar{x} = \frac{1}{N}\sum_i x_i, \qquad \bar{y} = \frac{1}{N}\sum_i y_i,$$

and the centered data

$$\mathbf{X}_c = \begin{bmatrix} (x_1 - \bar{x})^T \\ \vdots \\ (x_N - \bar{x})^T \end{bmatrix}, \qquad \boldsymbol{y}_c = \begin{bmatrix} y_1 - \bar{y} \\ \vdots \\ y_N - \bar{y} \end{bmatrix}.$$

Then the equations for the coefficients w and the intercept b are

$$\left(\frac{\mathbf{X}_c^T \mathbf{X}_c}{N} + \Lambda\right) w = \frac{\mathbf{X}_c^T \boldsymbol{y}_c}{N}, \qquad b = \bar{y} - \bar{x}^T w.$$

Scaling and regularization in Spark

Spark's parameter is $\rho = \texttt{regParam}$. For equal observation weights,

$$s_j = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_{ij} - \bar{x}_j)^2}, \qquad s_y = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \bar{y})^2},$$

where x_{ij} is the jth component of x_i, and $\bar{x}_j$ is the jth component of $\bar{x}$.

- `standardization=True` (default):

$$\Lambda = \frac{\rho}{s_y}\,\mathrm{diag}(s_1^2, \ldots, s_d^2).$$

- `standardization=False`:

$$\Lambda = \frac{\rho}{s_y} I_d.$$

Equations using the original data

Assume equal observation weights $a_i = 1$. Define

$$\mathbf{X} = \begin{bmatrix} x_1^T \\ \vdots \\ x_N^T \end{bmatrix} \in \mathbb{R}^{N \times d}, \qquad \boldsymbol{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix} \in \mathbb{R}^N.$$

Then

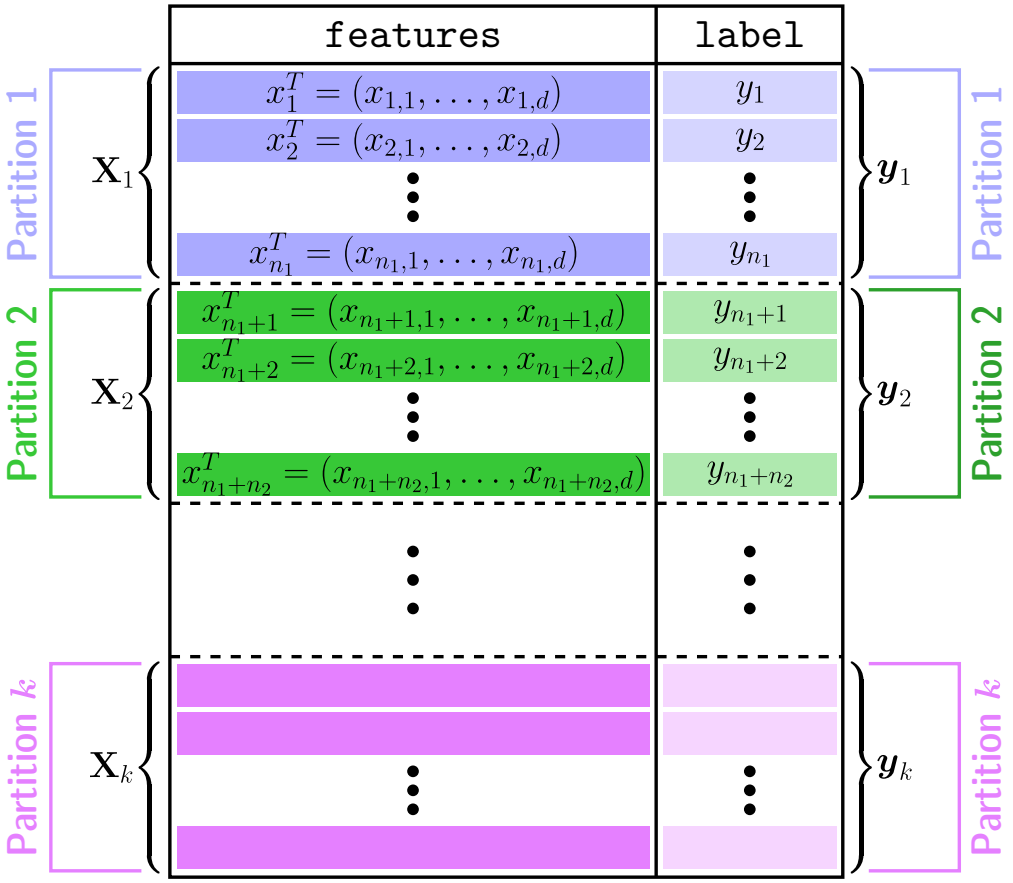
$$\frac{\mathbf{X}_c^T \mathbf{X}_c}{N} = \frac{\mathbf{X}^T \mathbf{X}}{N} - \bar{x}\bar{x}^T, \qquad \frac{\mathbf{X}_c^T \boldsymbol{y}_c}{N} = \frac{\mathbf{X}^T \boldsymbol{y}}{N} - \bar{x}\bar{y}.$$

where $\mathbf{1}$ is the vector of N ones. Then the equations for w and b are

$$\left(\frac{\mathbf{X}^T \mathbf{X}}{N} - \bar{x}\bar{x}^T + \Lambda \right) w = \frac{\mathbf{X}^T \boldsymbol{y}}{N} - \bar{x}\bar{y}, \qquad b = \bar{y} - \bar{x}^T w.$$

- This equation is to be solved on the driver.
- Quantities to be computed: N, $\bar{x}$, $\bar{y}$, $\mathbf{X}^T \mathbf{X}$, $\mathbf{X}^T \boldsymbol{y}$, $\boldsymbol{y}^T \boldsymbol{y}$.

Partitioning the training data



$$\mathbf{X} = \begin{bmatrix} \mathbf{X}_1 \\ \mathbf{X}_2 \\ \vdots \\ \mathbf{X}_k \end{bmatrix} \in \mathbb{R}^{N \times d},$$

$$\boldsymbol{y} = \begin{bmatrix} \boldsymbol{y}_1 \\ \boldsymbol{y}_2 \\ \vdots \\ \boldsymbol{y}_k \end{bmatrix} \in \mathbb{R}^{N}.$$

$$\mathbf{X}_p \in \mathbb{R}^{n_p \times d}, \qquad \boldsymbol{y}_p \in \mathbb{R}^{n_p}, \qquad p = 1, \ldots, k.$$

Distributed computation of the required quantities

Locally on partition p: Compute the row count n_p and

$$\mathbf{X}_p^T \mathbf{1}_{n_p}, \qquad \mathbf{1}_{n_p}^T \boldsymbol{y}_p, \qquad \mathbf{X}_p^T \mathbf{X}_p, \qquad \mathbf{X}_p^T \boldsymbol{y}_p, \qquad \boldsymbol{y}_p^T \boldsymbol{y}_p.$$

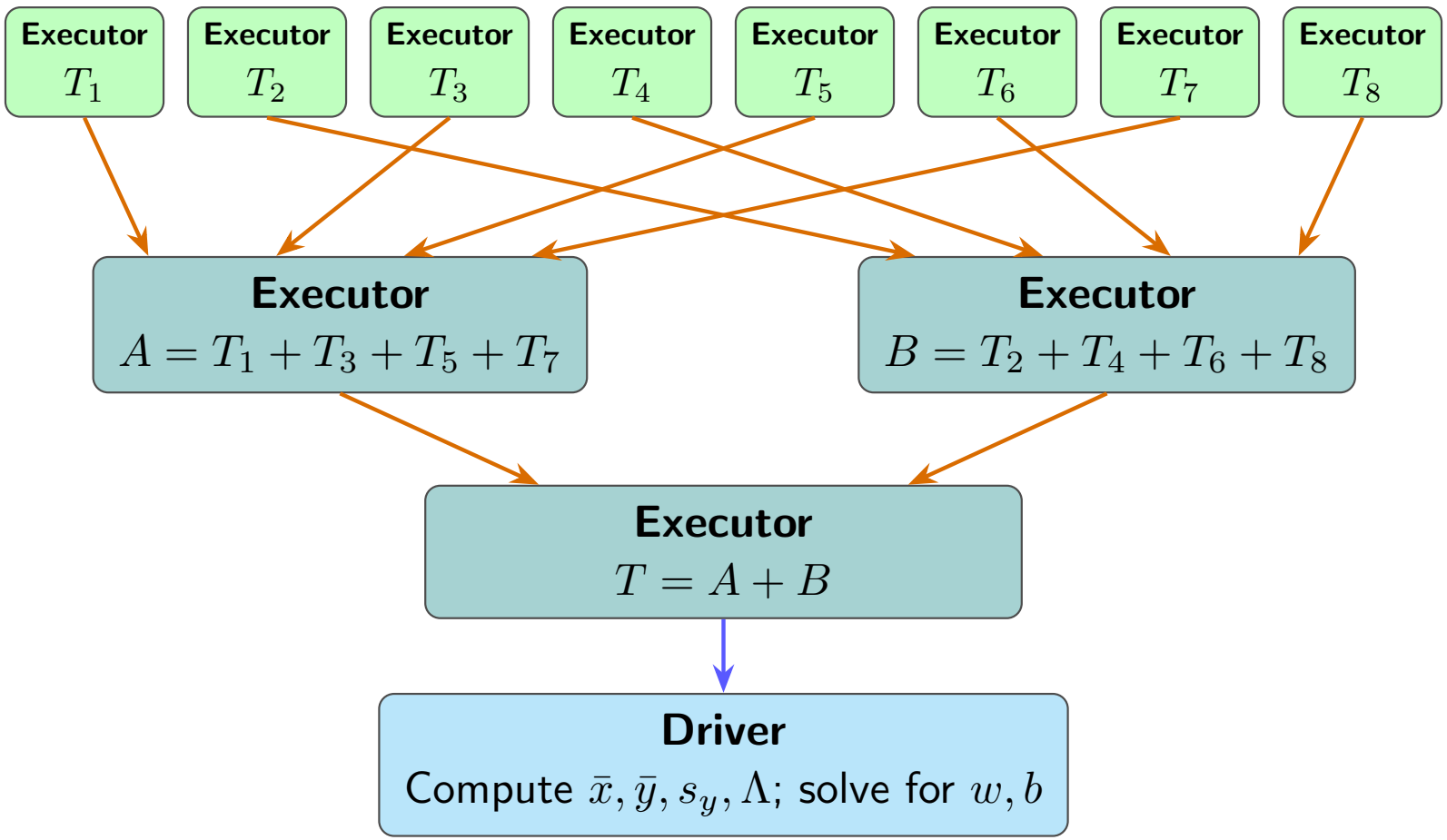
Aggregation: Add matching quantities from all partitions using `treeAggregate`:

- $N = n_1 + \cdots + n_k$.
- $\bar{x} = (\mathbf{X}_1^T \mathbf{1}_{n_1} + \cdots + \mathbf{X}_k^T \mathbf{1}_{n_k})/N$.
- $\bar{y} = (\mathbf{1}_{n_1}^T \boldsymbol{y}_1 + \cdots + \mathbf{1}_{n_k}^T \boldsymbol{y}_k)/N$.
- $\mathbf{X}^T \mathbf{X} = \mathbf{X}_1^T \mathbf{X}_1 + \cdots + \mathbf{X}_k^T \mathbf{X}_k$.
- $\mathbf{X}^T \boldsymbol{y} = \mathbf{X}_1^T \boldsymbol{y}_1 + \cdots + \mathbf{X}_k^T \boldsymbol{y}_k$.
- $\boldsymbol{y}^T \boldsymbol{y} = \boldsymbol{y}_1^T \boldsymbol{y}_1 + \cdots + \boldsymbol{y}_k^T \boldsymbol{y}_k$.

On the driver: Compute the means and $s_y^2 = \boldsymbol{y}^T \boldsymbol{y}/N - \bar{y}^2$; form Λ, then solve for w and b.

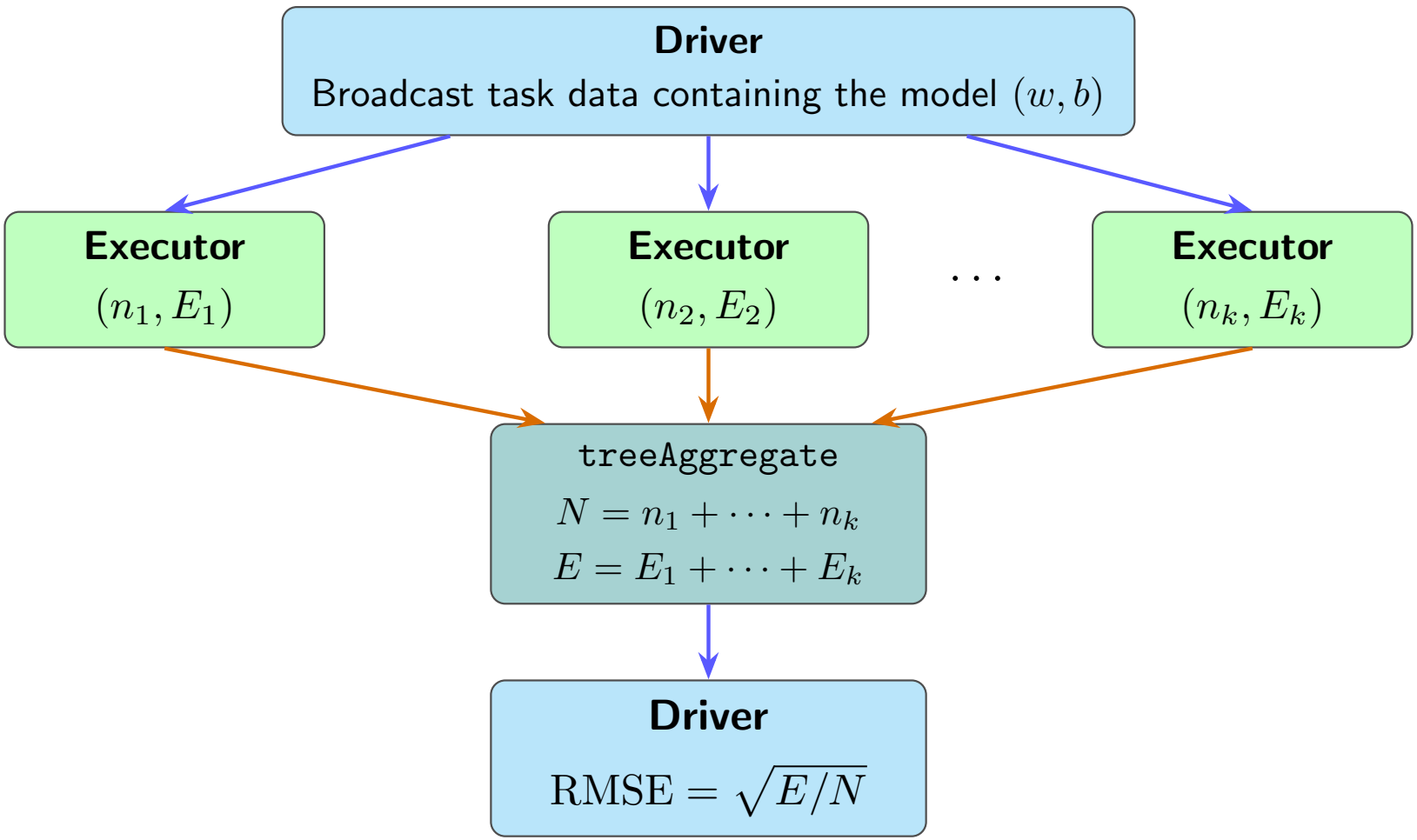
How Spark implements `treeAggregate`

$$T_p = \left(n_p,\ \mathbf{X}_p^T \mathbf{1}_{n_p},\ \mathbf{1}_{n_p}^T \boldsymbol{y}_p,\ \mathbf{X}_p^T \mathbf{X}_p,\ \mathbf{X}_p^T \boldsymbol{y}_p,\ \boldsymbol{y}_p^T \boldsymbol{y}_p \right).$$



Distributed prediction and evaluation

$$\widehat{\boldsymbol{y}}_p = \mathbf{X}_p w + b\mathbf{1}_{n_p}, \qquad \boldsymbol{e}_p = \boldsymbol{y}_p - \widehat{\boldsymbol{y}}_p, \qquad E_p = \boldsymbol{e}_p^T \boldsymbol{e}_p.$$



Numerical solver for linear regression

- When there are at most 4096 features ($d \leq 4096$), the driver solves the linear system

$$\left(\frac{\mathbf{X}^T\mathbf{X}}{N} - \bar{x}\bar{x}^T + \Lambda\right) w = \frac{\mathbf{X}^T\boldsymbol{y}}{N} - \bar{x}\bar{y}$$

  using Cholesky factorization. The computational cost is $O(d^3)$.
- When the matrix is singular, the driver retries the solve using quasi-Newton (L-BFGS) optimization on the aggregated statistics.
- When there are more than 4096 features, `solver="auto"` selects distributed L-BFGS for ridge regression.
- With ℓ^1 regularization, Spark uses the OWL-QN solver.

1 Introduction to MLlib

2 Supervised machine learning with MLlib

3 Distributed algorithms for machine learning
- Distributed algorithm for linear regression
- Distributed algorithm for alternating least squares

4 Cloud computing and Google cloud platform

5 Apache Spark with Managed Service on Google Cloud Platform

6 Summary

MovieLens: from ratings to recommendations

MovieLens 32M: 32 million ratings from 200,948 users on 87,585 movies, collected by GroupLens Research.

`ratings.csv`

userId	movieId	rating	timestamp
1	17	4.0	944249077
1	25	1.0	944250228
⋮	⋮	⋮	⋮
100297	5060	4.0	952049586
100298	1	2.0	832419939
100298	10	3.0	832419939
⋮	⋮	⋮	⋮
200948	80463	3.5	1350423800
200948	87304	4.5	1350423523

`movies.csv`

movieId	title	genres
1	Toy Story (1995)	Adventure|Animation|...
2	Jumanji (1995)	Adventure|Children|...
⋮	⋮	⋮
90	The journey o...	Drama
92	Mary Reilly (...	Drama|Horror|Thriller
⋮	⋮	⋮
292753	Orca (2003)	Drama
292755	The Angry Bre...	Drama
292757	Race to the S...	Action|Adventure|...

`https://grouplens.org/datasets/movielens/32m/`

Most user-movie pairs have no rating.

Load MovieLens and fit an ALS model

```
from pyspark.ml.recommendation import ALS
from pyspark.ml.evaluation import RegressionEvaluator

# Load data and train the ALS model
ratings = spark.read.option("header", True)\
    .option("inferSchema", True).csv(ratings.csv)
train, test = ratings.randomSplit([0.8, 0.2])
als = ALS(userCol="userId", itemCol="movieId", ratingCol="rating",
          rank=10, maxIter=10, regParam=0.1,
          implicitPrefs=False, nonnegative=False,
          coldStartStrategy="drop")
model = als.fit(train)

# Evaluate the model by computing the RMSE on the test data
predictions = model.transform(test)
evaluator = RegressionEvaluator(metricName="rmse",
                                labelCol="rating",
                                predictionCol="prediction")
print("Test RMSE:", evaluator.evaluate(predictions))
```

Make recommendations

Recommend the top 5 movies for 3 random users from the training set.

```
from pyspark.sql.functions import explode, col, rand

movies = spark.read.option("header", True)\
    .option("inferSchema", True).csv("movies.csv")
users = train.select("userId").distinct().orderBy(rand()).limit(3)
recommendations = model.recommendForUserSubset(users, 5)\
    .select("userId", explode("recommendations").alias("rec"))\
    .select("userId", col("rec.movieId").alias("movieId"),
            col("rec.rating").alias("prediction"))\
    .join(movies, "movieId")\
    .select("userId", "title", "genres", "prediction")
recommendations.show()
```

The recommendations may contain movies that the user has already rated. Can you recommend only unseen movies?

Ratings and the recommendation task

Let r_{ui} be the rating given by user u to item i.

	Item 1	Item 2	Item 3	Item 4	Item 5	Item 6
User 1	–	2	–	5	3	–
User 2	3	**5**	–	–	–	4
User 3	4	–	5	–	2	–
User 4	–	4	1	–	–	5

- $r_{22} = 5$: user 2 gave item 2 a rating of 5.
- User 2 has not rated item 4. We want to predict this rating, denoted by $\widehat{r}_{24}$.
- Let Ω be the set of pairs with known ratings. For example, $(2, 2) \in \Omega$, $(3, 1) \in \Omega$, but $(2, 4) \notin \Omega$.

Predict a rating using two short vectors

Prediction: Give each user a vector

$$p_u = (p_{u1}, p_{u2}, \ldots, p_{ud}) \in \mathbb{R}^d$$

and each item a vector

$$q_i = (q_{i1}, q_{i2}, \ldots, q_{id}) \in \mathbb{R}^d.$$

Our prediction is the inner product of the two vectors:

$$\widehat{r}_{ui} = p_u^T q_i = p_{u1}q_{i1} + \cdots + p_{ud}q_{id}.$$

The hyperparameter d is `rank` in Spark.

Training: Choose the vectors p_u and q_i to make

$$\widehat{r}_{ui} \approx r_{ui}, \qquad \forall (u, i) \in \Omega.$$

Example

Ratings:

r_{ui}	$i=1$	$i=2$	$i=3$	$i=4$	$i=5$	$i=6$
$u=1$	–	2	–	5	3	–
$u=2$	3	5	–	–	–	4
$u=3$	4	–	5	–	2	–
$u=4$	–	4	1	–	–	5

Example with $d=2$: Consider these user and item vectors:

$$p_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \qquad p_2 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \qquad q_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \qquad q_4 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

- Known ratings:

$$\widehat{r}_{14} = p_1^T q_4 = 5 = r_{14},$$
$$\widehat{r}_{21} = p_2^T q_1 = 3 = r_{21}.$$

- Unseen items:

$$\widehat{r}_{11} = p_1^T q_1 = 3,$$
$$\widehat{r}_{24} = p_2^T q_4 = 4.$$

Learn the vectors from observed ratings

Fit the known ratings:

$$\min_{\{p_u\},\{q_i\}} \frac{1}{2} \sum_{(u,i)\in\Omega} (r_{ui} - p_u^T q_i)^2,$$

Learn the vectors from observed ratings

Fit the known ratings:

$$\min_{\{p_u\},\{q_i\}} \frac{1}{2} \sum_{(u,i)\in\Omega} (r_{ui} - p_u^T q_i)^2 + \frac{\lambda}{2} \left(\sum_u n_u \|p_u\|^2 + \sum_i m_i \|q_i\|^2 \right),$$

where

- n_u: number of ratings given by user u.
- m_i: number of ratings received by item i.

Learn the vectors from observed ratings

Fit the known ratings:

$$\min_{\{p_u\},\{q_i\}} \frac{1}{2} \sum_{(u,i)\in\Omega} (r_{ui} - p_u^T q_i)^2 + \frac{\lambda}{2} \left(\sum_u n_u \|p_u\|^2 + \sum_i m_i \|q_i\|^2 \right),$$

where

- n_u: number of ratings given by user u.
- m_i: number of ratings received by item i.

However, this optimization problem does not admit a closed-form solution.

Alternating least squares

1. Initialize the factor vectors randomly.
2. **Item step:** Fix all user vectors p_u and update each item vector q_i by solving

$$\min_{\{q_i\}} \frac{1}{2} \sum_{(u,i)\in\Omega} (r_{ui} - p_u^T q_i)^2 + \frac{\lambda}{2} \sum_i m_i \|q_i\|^2. \tag{1}$$

3. **User step:** Fix the updated item vectors q_i and update each user vector p_u by solving

$$\min_{\{p_u\}} \frac{1}{2} \sum_{(u,i)\in\Omega} (r_{ui} - p_u^T q_i)^2 + \frac{\lambda}{2} \sum_u n_u \|p_u\|^2. \tag{2}$$

4. Repeat the two steps for `maxIter` iterations.

Both (1) and (2) admit closed-form solutions!

Each update is a small least-squares problem

Assume that ALS uses B_I item blocks and B_U user blocks.

- **Item step:** Equation for solving q_i:

$$\left(\sum_{u:(u,i)\in\Omega} p_u p_u^T + \lambda m_i I_d \right) q_i = \sum_{u:(u,i)\in\Omega} p_u r_{ui}$$

  - Item block 0 solves q_i for $i \in I_0 = \{B_I, 2B_I, 3B_I, \ldots\}$;
  - Item block 1 solves q_i for $i \in I_1 = \{1, B_I + 1, 2B_I + 1, \ldots\}$;
  - ...

  Solving q_i for $i \in I_k$ requires all p_u for users u who rated item i and all r_{ui} for those ratings.

Each update is a small least-squares problem

Assume that ALS uses B_I item blocks and B_U user blocks.

- **User step:** Equation for solving p_u:

$$\left(\sum_{i:(u,i)\in\Omega} q_i q_i^T + \lambda n_u I_d \right) p_u = \sum_{i:(u,i)\in\Omega} q_i r_{ui}$$

  - User block 0 solves p_u for $u \in U_0 = \{B_U, 2B_U, 3B_U, \ldots\}$;
  - User block 1 solves p_u for $u \in U_1 = \{1, B_U + 1, 2B_U + 1, \ldots\}$;
  - ...

  Solving p_u for $u \in U_k$ requires all q_i for items i that user u rated and all r_{ui} for those ratings.

Which vectors and ratings does each block need?

Two user blocks ($B_U = 2$) and three item blocks ($B_I = 3$).

	Item 1	Item 2	Item 3	Item 4	Item 5	Item 6
User 1	–	2	–	5	3	–
User 2	3	5	–	–	–	4
User 3	4	–	5	–	2	–
User 4	–	4	1	–	–	5

Item step: fix user vectors and update item vectors.

Item block 0: $I_0 = \{3, 6\}$

Update q_3, q_6.

Needs p_2, p_3, p_4.

$r_{33} = 5, \quad r_{43} = 1,$

$r_{26} = 4, \quad r_{46} = 5.$

Item block 1: $I_1 = \{1, 4\}$

Update q_1, q_4.

Needs p_1, p_2, p_3.

$r_{21} = 3, \quad r_{31} = 4,$

$r_{14} = 5.$

Item block 2: $I_2 = \{2, 5\}$

Update q_2, q_5.

Needs p_1, p_2, p_3, p_4.

$r_{12} = 2, \quad r_{22} = 5, \quad r_{42} = 4,$

$r_{15} = 3, \quad r_{35} = 2.$

Which vectors and ratings does each block need?

Two user blocks ($B_U = 2$) and three item blocks ($B_I = 3$).

	Item 1	Item 2	Item 3	Item 4	Item 5	Item 6
User 1	–	2	–	5	3	–
User 2	3	5	–	–	–	4
User 3	4	–	5	–	2	–
User 4	–	4	1	–	–	5

User step: fix item vectors and update user vectors.

User block 0: $U_0 = \{2, 4\}$

Update p_2, p_4.

Needs q_1, q_2, q_3, q_6.

$r_{21} = 3, \quad r_{22} = 5, \quad r_{26} = 4,$

$r_{42} = 4, \quad r_{43} = 1, \quad r_{46} = 5.$

User block 1: $U_1 = \{1, 3\}$

Update p_1, p_3.

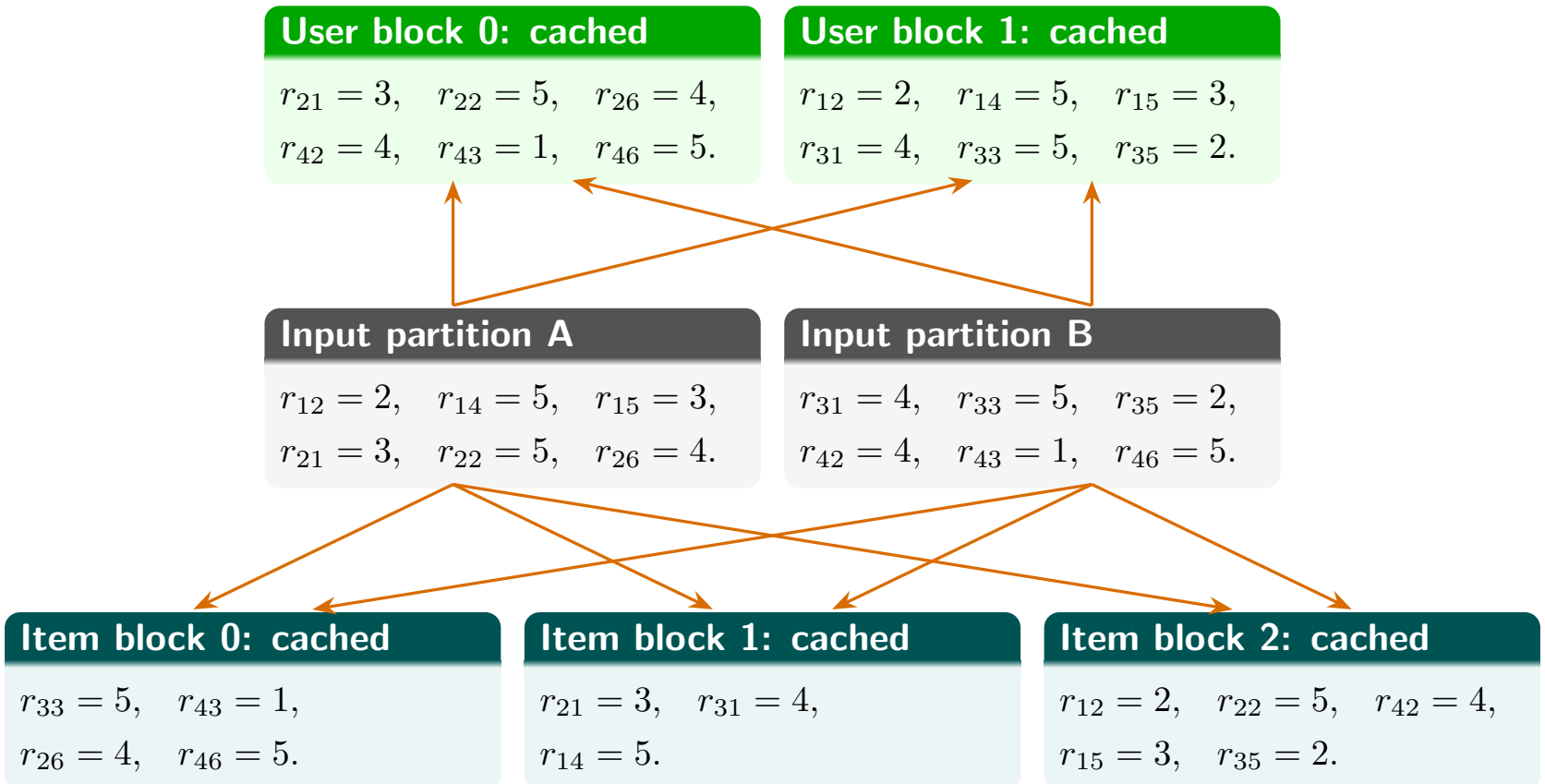
Needs q_1, q_2, q_3, q_4, q_5.

$r_{12} = 2, \quad r_{14} = 5, \quad r_{15} = 3,$

$r_{31} = 4, \quad r_{33} = 5, \quad r_{35} = 2.$

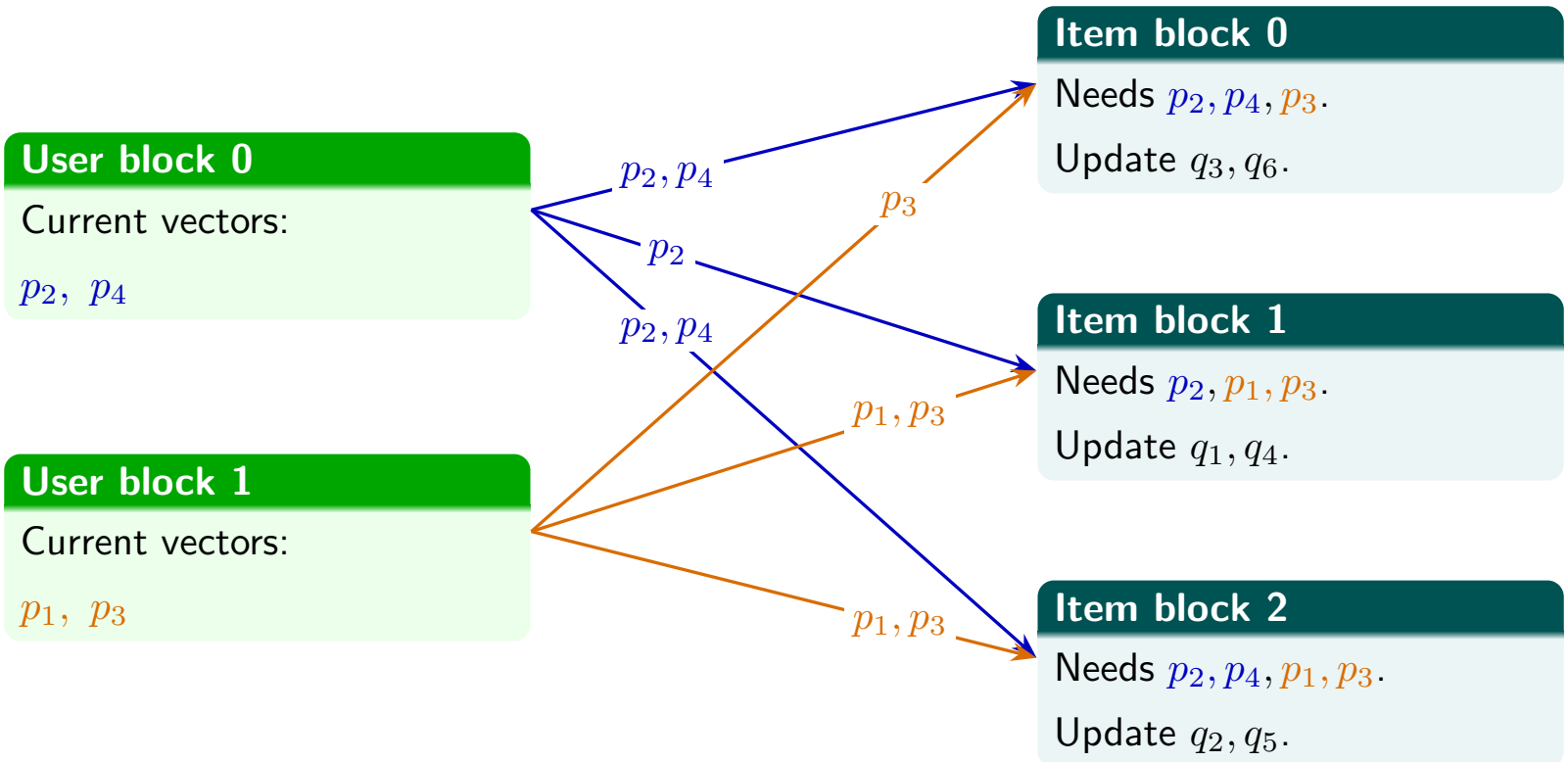
Initialization

Shuffle and cache the ratings into user and item blocks:



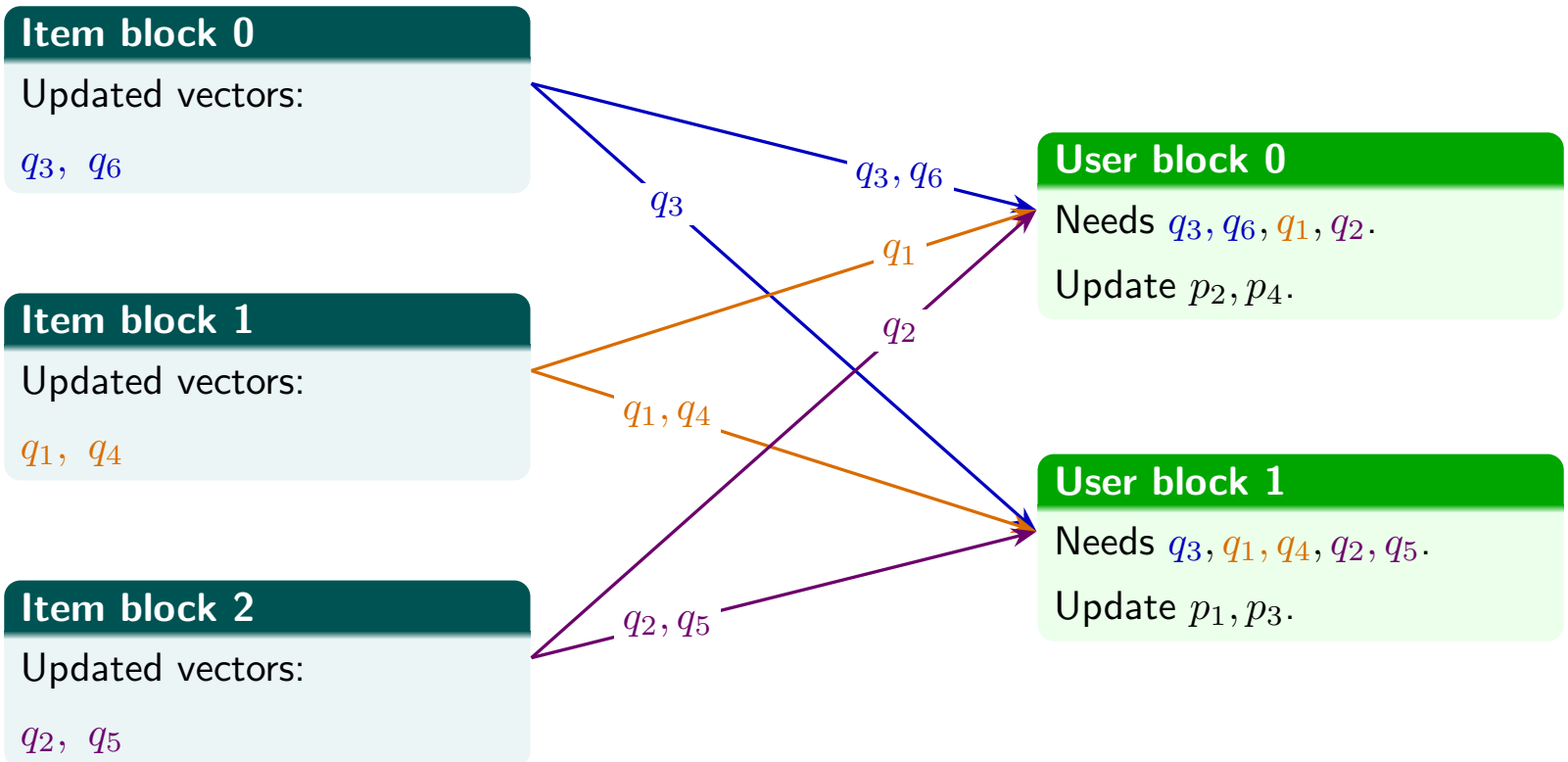
Before each item step

Shuffle: send current p_u to the item blocks that need them.



Before each user step

Shuffle: send updated q_i to the user blocks that need them.



Overall algorithm: initialization

Settings: B_U user blocks, B_I item blocks, rank d, regularization parameter λ, and T iterations.

Input: Each partition holds observed ratings r_{ui} for a subset of Ω.

1. Assign blocks: Put user u in block $u \bmod B_U$ and item i in block $i \bmod B_I$.
2. Redistribute and cache ratings: Shuffle the ratings such that each item and user block contains the ratings.
3. Count the observed ratings: For each user and item, compute

$$n_u = |\{i : (u, i) \in \Omega\}|, \qquad m_i = |\{u : (u, i) \in \Omega\}|.$$

4. Prepare the transfers: Record which source vectors each destination block needs; reuse these index lists in every iteration.
5. Initialize the vectors: Choose all $p_u, q_i \in \mathbb{R}^d$ randomly.

Overall algorithm: alternating updates

Repeat the following four steps for T iterations:

1. Shuffle user vectors: Send the required p_u to each item block.
2. Update item vectors in parallel across blocks: For each $i \in I_b$, use cached ratings and received p_u to solve

$$\left(\sum_{u:(u,i)\in\Omega} p_u p_u^T + \lambda m_i I_d \right) q_i = \sum_{u:(u,i)\in\Omega} r_{ui} p_u.$$

3. Shuffle item vectors: Send the required updated q_i to each user block.
4. Update user vectors in parallel across blocks: For each $u \in U_a$, use cached ratings and received q_i to solve

$$\left(\sum_{i:(u,i)\in\Omega} q_i q_i^T + \lambda n_u I_d \right) p_u = \sum_{i:(u,i)\in\Omega} r_{ui} q_i.$$

1 Introduction to MLlib

2 Supervised machine learning with MLlib

3 Distributed algorithms for machine learning

4 Cloud computing and Google cloud platform

5 Apache Spark with Managed Service on Google Cloud Platform

6 Summary

What is cloud computing?

Definition of cloud computing:

- Google says[2]:

  Cloud computing is the on-demand availability of computing resources (such as storage and infrastructure), as services over the internet. It eliminates the need for individuals and businesses to self-manage physical resources themselves, and only pay for what they use.

[2] https://cloud.google.com/learn/what-is-cloud-computing

What is cloud computing?

Definition of cloud computing:

- Amazon says[2]:

  Cloud computing is the on-demand delivery of IT resources over the Internet with pay-as-you-go pricing. Instead of buying, owning, and maintaining physical data centers and servers, you can access technology services, such as computing power, storage, and databases, on an as-needed basis from a cloud provider.

[2] https://aws.amazon.com/what-is-cloud-computing

What is cloud computing?

Definition of cloud computing:

- Microsoft says[2]:

  Cloud computing is the delivery of computing services–including servers, storage, databases, networking, software, analytics, and intelligence–over the internet ("the cloud") to offer faster innovation, flexible resources, and economies of scale. You typically pay only for cloud services you use, helping you lower your operating costs, run your infrastructure more efficiently, and scale as your business needs change.

[2] https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-cloud-computing

What is cloud computing?

Definition of cloud computing:

- Microsoft says[2]:

  Cloud computing is the delivery of computing services–including servers, storage, databases, networking, software, analytics, and intelligence–over the internet ("the cloud") to offer faster innovation, flexible resources, and economies of scale. You typically pay only for cloud services you use, helping you lower your operating costs, run your infrastructure more efficiently, and scale as your business needs change.

Popular cloud computing platforms:

Google Cloud



Amazon Web Services



Microsoft Azure

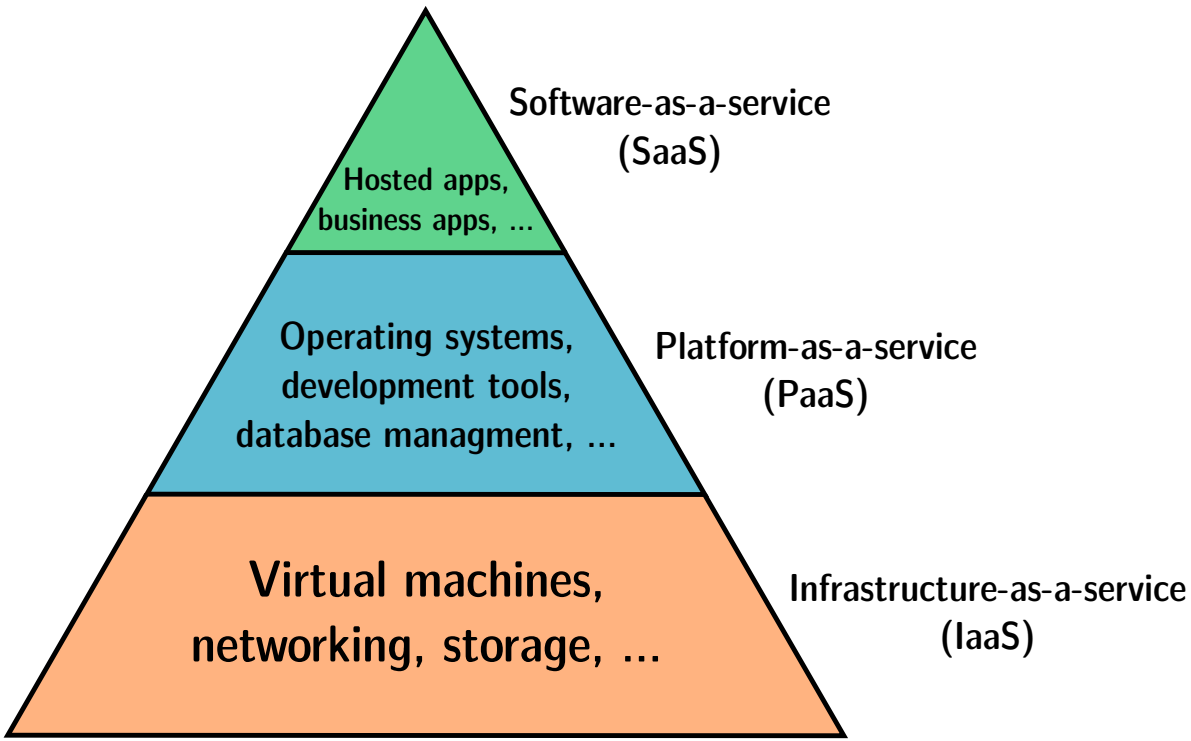


[2] https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-cloud-computing

Cloud computing models[3]

Infrastructure as a service (IaaS):

- IaaS delivers on-demand infrastructure resources to organizations via the cloud, such as compute, storage, networking, and virtualization.
- Customers are responsible for the operating system, middleware, virtual machines, and any apps or data.

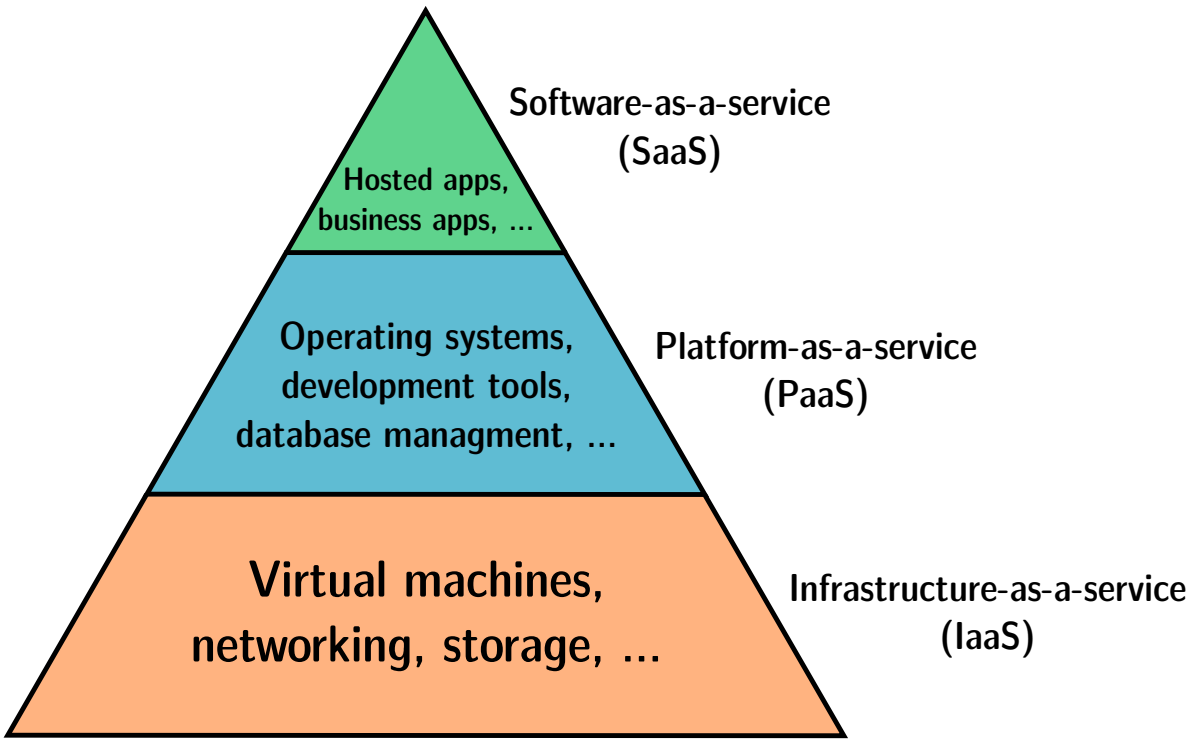


[3] https://cloud.google.com/learn/paas-vs-iaas-vs-saas

Cloud computing models[3]

Platform as a service (PaaS):

- PaaS delivers and manages all the hardware and software resources to develop applications through the cloud.
- Customers still have to write the code and manage their data and applications.

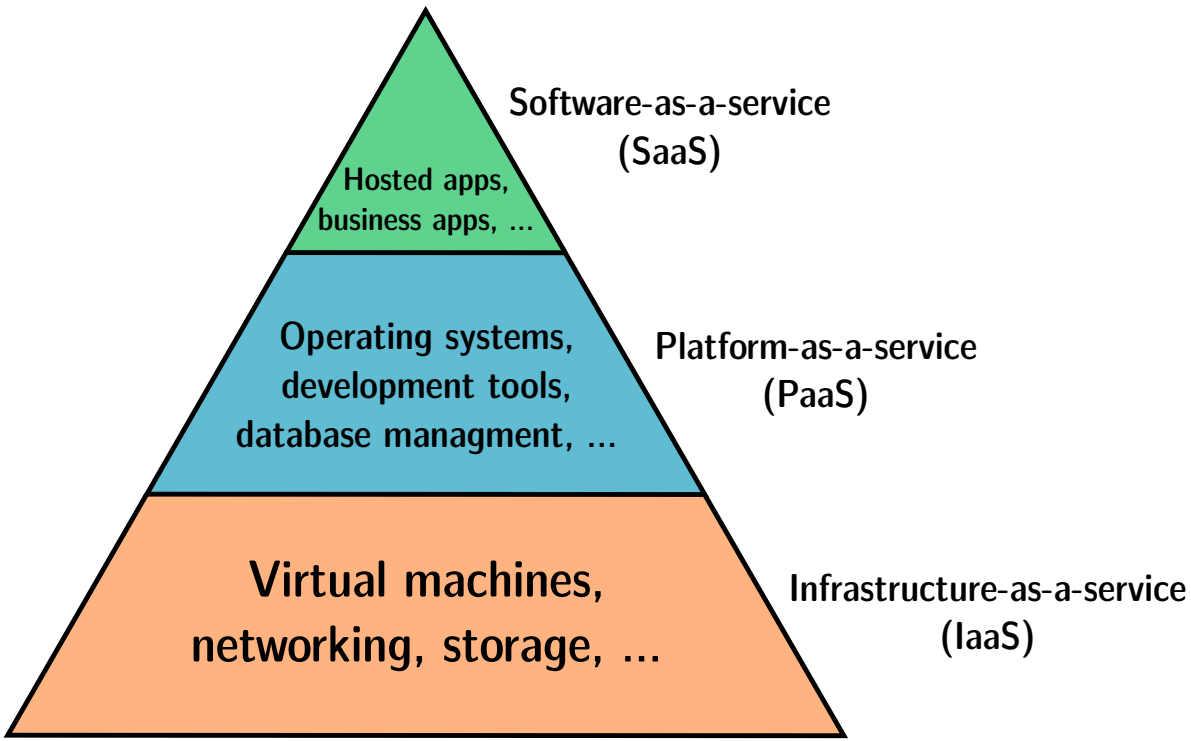


[3] https://cloud.google.com/learn/paas-vs-iaas-vs-saas

Cloud computing models[3]

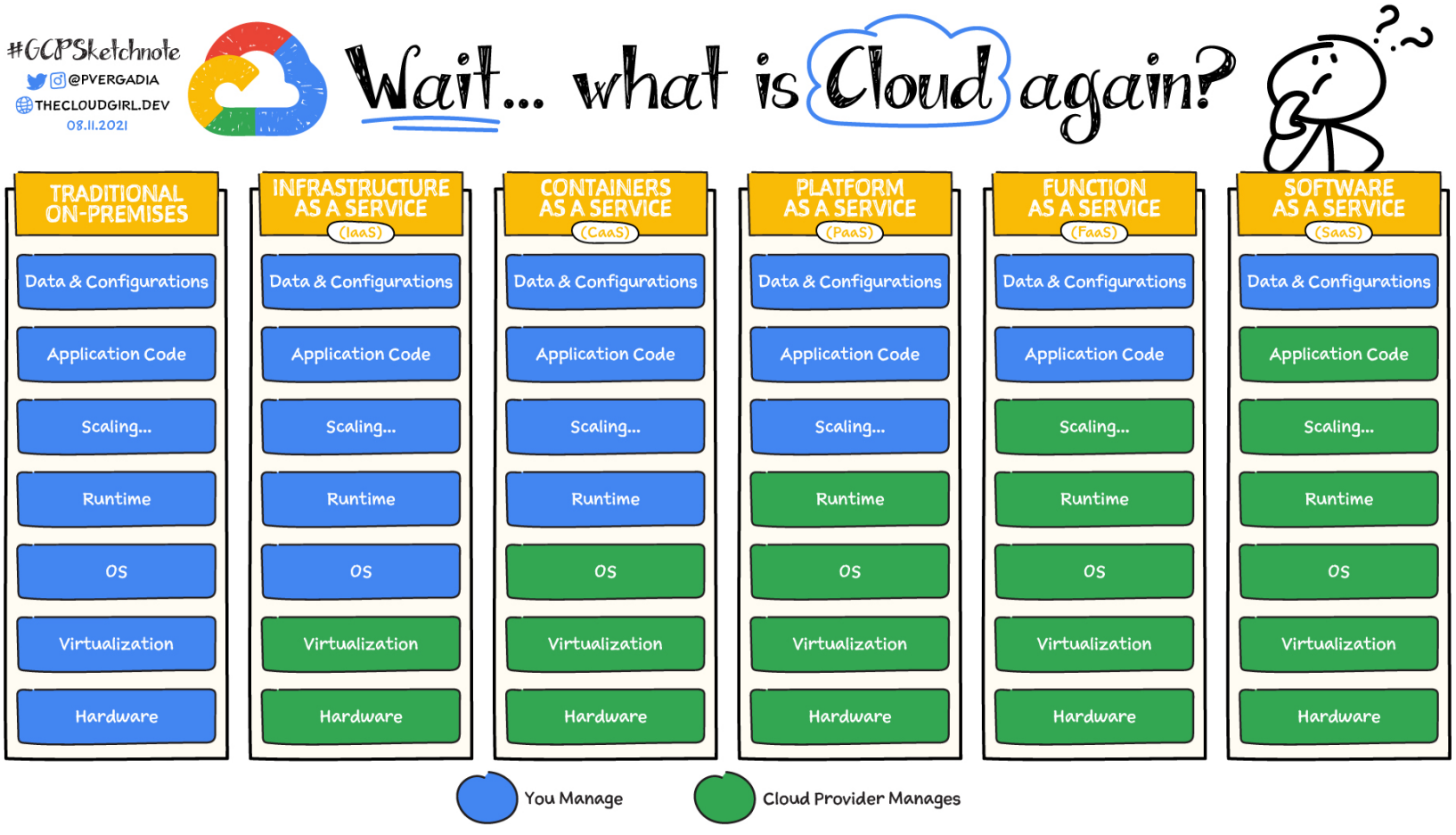
Software as a service (SaaS):

- SaaS provides the entire application stack, delivering an entire cloud-based application that customers can access and use.
- Customers are only responsible for taking care of their own data and configurations.



[3] https://cloud.google.com/learn/paas-vs-iaas-vs-saas

Others



From `https://cloud.google.com/learn/paas-vs-iaas-vs-saas`

Google cloud platform

Google (so do Microsoft and Amazon) offers solutions to all the three cloud computing models:

- **IaaS:**

Compute Engine  Cloud Storage 

- **PaaS:**

Cloud Run  App Engine  Data Analytics 

- **SaaS:**

Google Workspace  Looker 

Managed Service for Apache Spark

Managed Service for Apache Spark is a fully managed and highly scalable service for running 30+ open source tools and frameworks.

Managed Service for Apache Spark components:

- Apache Spark
- Apache Hadoop
- Apache Pig
- Apache Hive
- Anaconda
- Docker
- Jupyter Notebook
- Presto
- Zeppelin Notebook
- ...

1 Introduction to MLlib

2 Supervised machine learning with MLlib

3 Distributed algorithms for machine learning

4 Cloud computing and Google cloud platform

5 Apache Spark with Managed Service on Google Cloud Platform

6 Summary

1 Introduction to MLlib

2 Supervised machine learning with MLlib

3 Distributed algorithms for machine learning

4 Cloud computing and Google cloud platform

5 Apache Spark with Managed Service on Google Cloud Platform
- Basic usage
- Use clusters with GPUs
- Optimization

6 Summary

Managed Service for Apache Spark pricing

Managed Service for Apache Spark uses Google Compute Engine (GCE) or Google Kubernetes Engine (GKE) as its infrastructure. For simplicity, we will only look at managed Spark clusters on GCE.

[4] https://cloud.google.com/compute/all-pricing

[5] https://cloud.google.com/products/managed-service-for-apache-spark/pricing

Managed Service for Apache Spark pricing

Managed Service for Apache Spark uses Google Compute Engine (GCE) or Google Kubernetes Engine (GKE) as its infrastructure. For simplicity, we will only look at managed Spark clusters on GCE.

To use Managed Service for Apache Spark, we need to pay for

- **Compute Engine:**[4]
  - The price depends on the machine type, disk size, network usage, etc.
  - Use `https://cloud.google.com/products/calculator` to estimate the cost.
- **Managed Spark clusters:**[5]
  - The price depends on the number of virtual CPUs and the duration of a cluster.

[4] `https://cloud.google.com/compute/all-pricing`

[5] `https://cloud.google.com/products/managed-service-for-apache-spark/pricing`

Managed Service for Apache Spark pricing

Managed Service for Apache Spark uses Google Compute Engine (GCE) or Google Kubernetes Engine (GKE) as its infrastructure. For simplicity, we will only look at managed Spark clusters on GCE.

To use Managed Service for Apache Spark, we need to pay for

- **Compute Engine:**[4]
  - The price depends on the machine type, disk size, network usage, etc.
  - Use `https://cloud.google.com/products/calculator` to estimate the cost.
- **Managed Spark clusters:**[5]
  - The price depends on the number of virtual CPUs and the duration of a cluster.

New customers get $300 in free credit to try Google Cloud products!

[4] `https://cloud.google.com/compute/all-pricing`

[5] `https://cloud.google.com/products/managed-service-for-apache-spark/pricing`

Google Cloud Command Line Interface (gcloud CLI)

The Google Cloud CLI is a set of tools to create and manage Google Cloud resources, including

- Compute Engine virtual machine instances and other resources
- Cloud SQL instances
- Google Kubernetes Engine clusters
- Managed Spark clusters and jobs
- Cloud DNS managed zones and record sets
- Cloud Deployment Manager deployments

Check `https://docs.cloud.google.com/sdk/docs/install` for instructions to install gcloud CLI.

Check `https://docs.cloud.google.com/sdk/docs/cheatsheet` for the gcloud CLI cheat sheet.

Basic setup

1. Log in to your Google account:

```
$ gcloud auth login
```

2. Create a project:

```
$ gcloud projects create [PROJECT NAME]
$ gcloud config set project [PROJECT NAME]
```

3. Link to a billing account:

```
$ gcloud billing accounts list
$ gcloud billing projects link [PROJECT NAME] --billing-account=[ACCOUNT ID]
```

4. Enable Google APIs:

```
$ gcloud services enable dataproc.googleapis.com
$ gcloud services enable cloudresourcemanager.googleapis.com
```

5. Set permissions:

```
$ gcloud iam service-accounts list
$ gcloud projects add-iam-policy-binding [PROJECT NAME] \
  --member="serviceAccount:[EMAIL]" --role="roles/dataproc.worker"
```

Create a cluster

```
$ gcloud compute networks subnets update default \
  --region=[REGION] --enable-private-ip-google-access
$ gcloud dataproc clusters create [CLUSTER NAME] \
  --region [REGION] --image-version [IMAGE VERSION] \
  --master-machine-type [MASTER MACHINE TYPE] \
  --master-boot-disk-type [MASTER DISK TYPE] --master-boot-disk-size [DISK SIZE] \
  --num-workers [NUMBER OF WORKERS] \
  --worker-machine-type [WORKER MACHINE TYPE] \
  --worker-boot-disk-type [WORKER DISK TYPE] --worker-boot-disk-size [DISK SIZE] \
  --enable-component-gateway --optional-components JUPYTER
```

- `[REGION]` defines the location of virtual machines.[6]
- `[MASTER/WORKER MACHINE TYPE]`, `[MASTER/WORKER DISK TYPE]` and `[MASTER/WORKER DISK SIZE]` define the configurations of virtual machines.[7]
- `[IMAGE VERSION]` defines the verions of the operating systems and software packages.[8]

[6] https://cloud.google.com/compute/docs/regions-zones
[7] https://cloud.google.com/compute/docs/general-purpose-machines
[8] https://docs.cloud.google.com/managed-spark/docs/concepts/versioning/image-version-lists

Run PySpark jobs

1. Copy local files to virtual machines:

```
$ gcloud compute instances list
$ gcloud compute scp [LOCAL FILES] [VM NAME]:[REMOTE PATH]
```

2. SSH to virtual machines:

```
$ gcloud compute ssh [VM NAME]
```

3. Use HDFS and PySpark:

```
$ hadoop fs -copyFromLocal flights.csv /user/spark/
$ pyspark
>>> df = spark.read.csv("hdfs:///user/spark/flights.csv",
...                     header=True, inferSchema=True)
>>> from pyspark.sql.functions import max, min, avg, col
>>> df.groupBy("AIRLINE").agg(
...     max("ARRIVAL_DELAY").alias("MAX_ARR_DELAY"),
...     min("ARRIVAL_DELAY").alias("MIN_ARR_DELAY"),
...     avg("ARRIVAL_DELAY").alias("AVG_ARR_DELAY")
... ).sort(col("AVG_ARR_DELAY").desc()).show()
>>> quit()
```

4. Exit from the virtual machine:

```
$ exit
```

Submit PySpark jobs

Alternatively, we can write our PySpark jobs into a python script and submit it to the cluster using gcloud CLI:

```
$ gcloud dataproc clusters list --region=[REGION]
$ gcloud dataproc jobs submit pyspark --cluster=[CLUSTER NAME] \
                                      --region=[REGION] [PYSPARK SCRIPT]
```

A sample pyspark script:

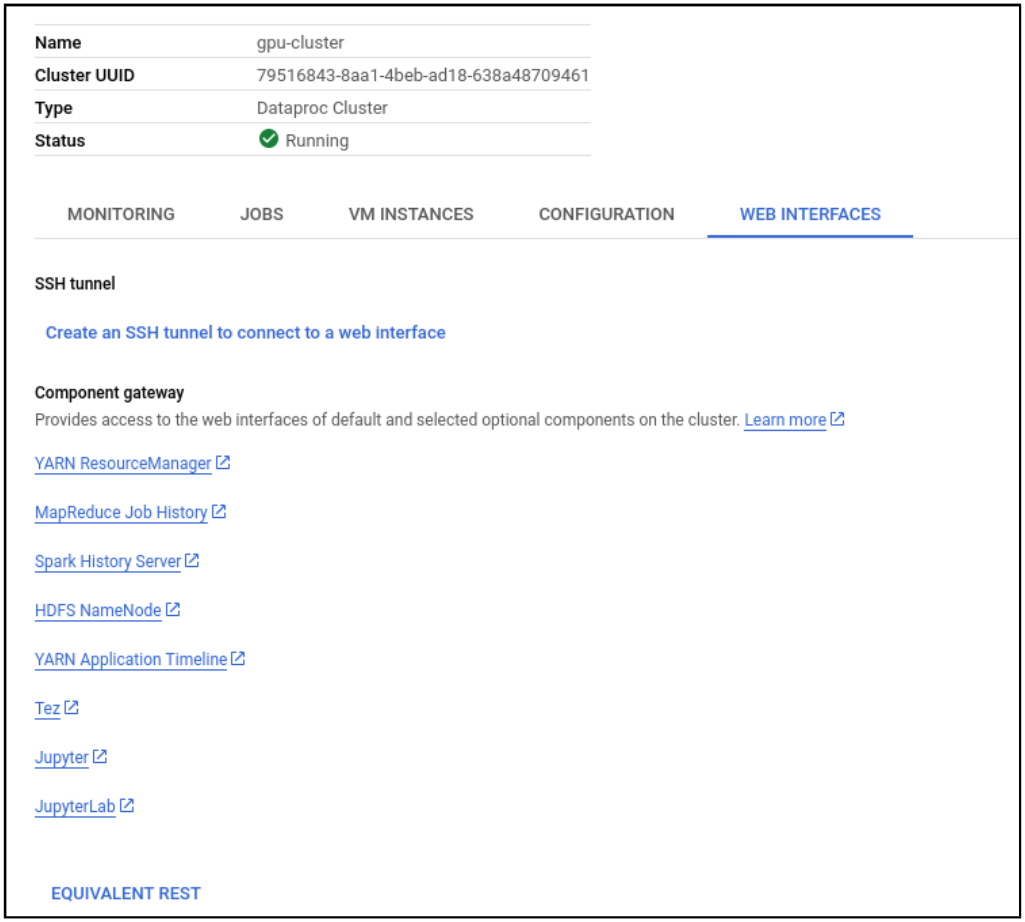
```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("Managed Spark Demo").getOrCreate()
df = spark.read.csv("hdfs:///user/spark/flights.csv", inferSchema=True, header=True)

from pyspark.sql.functions import max, min, avg, col
airlineDelay = df.groupBy("AIRLINE").agg(
        max("ARRIVAL_DELAY").alias("MAX_ARR_DELAY"),
        min("ARRIVAL_DELAY").alias("MIN_ARR_DELAY"),
        avg("ARRIVAL_DELAY").alias("AVG_ARR_DELAY")
).sort(col("AVG_ARR_DELAY").desc())

airlineDelay.write.mode("overwrite").csv("hdfs:///user/spark/airlineDelay.csv",
                                         header=True)
```

Use web interfaces

Menu ≡ > Managed Spark > Clusters > [Cluster Name] > WEB INTERFACES

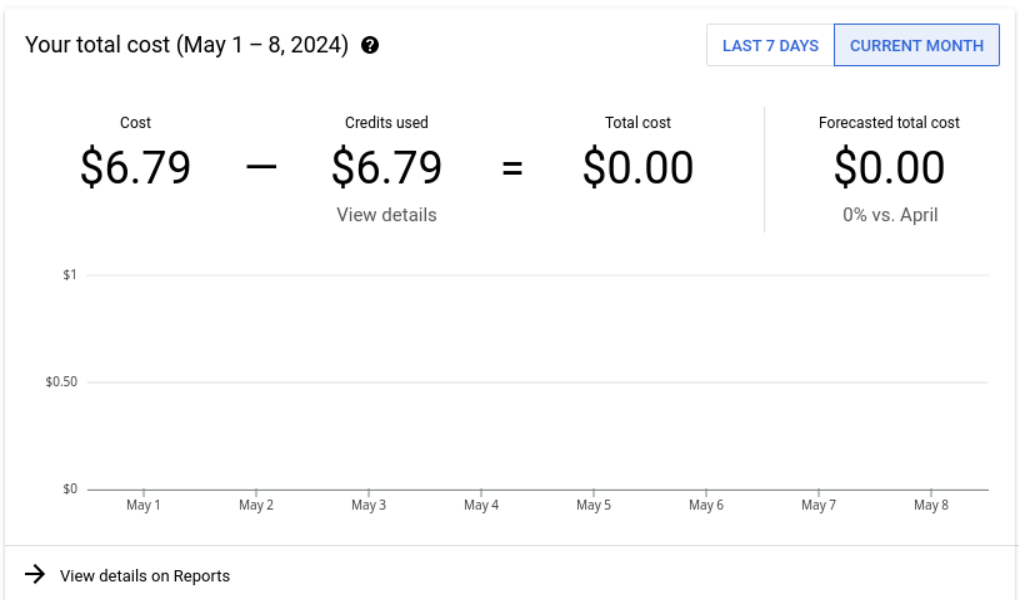


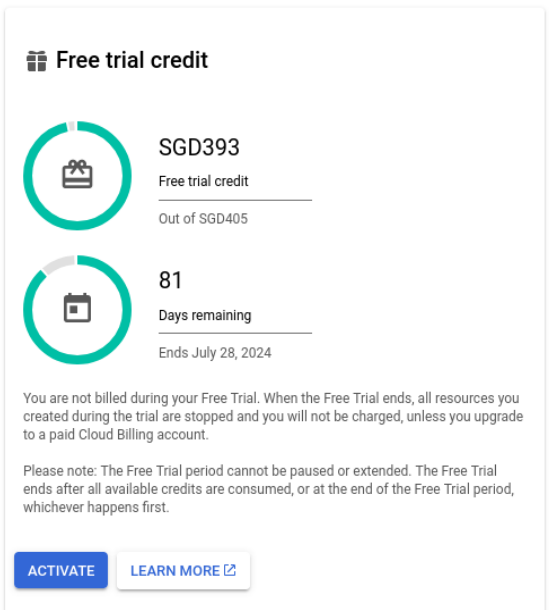
Delete clusters

Do remember to delete clusters to avoid being charged!

```
$ gcloud dataproc clusters delete [CLUSTER NAME] --region=[REGION]
```

Check balance: Menu ≡ > Billing > [Choose account]





1 Introduction to MLlib

2 Supervised machine learning with MLlib

3 Distributed algorithms for machine learning

4 Cloud computing and Google cloud platform

5 Apache Spark with Managed Service on Google Cloud Platform
- Basic usage
- Use clusters with GPUs
- Optimization

6 Summary

cuDF for Apache Spark

cuDF for Apache Spark leverages GPUs to accelerate processing via the **RAPIDS cuDF library**, which is a GPU DataFrame library for loading, joining, aggregating, filtering, and otherwise manipulating data.

cuDF for Apache Spark

cuDF for Apache Spark leverages GPUs to accelerate processing via the **RAPIDS cuDF library**, which is a GPU DataFrame library for loading, joining, aggregating, filtering, and otherwise manipulating data.

- The accelerator acts as a plugin of Apache Spark.
- It requires no API change from the user.
- The plugin replaces operations it supports with GPU accelerated versions. If an operation is not supported it will fall back to using the Spark CPU version.
- The plugin cannot accelerate operations that manipulate RDDs directly.

cuDF for Apache Spark

cuDF for Apache Spark leverages GPUs to accelerate processing via the **RAPIDS cuDF library**, which is a GPU DataFrame library for loading, joining, aggregating, filtering, and otherwise manipulating data.

- The accelerator acts as a plugin of Apache Spark.
- It requires no API change from the user.
- The plugin replaces operations it supports with GPU accelerated versions. If an operation is not supported it will fall back to using the Spark CPU version.
- The plugin cannot accelerate operations that manipulate RDDs directly.

Setting up cuDF for Apache Spark is straightforward with Managed Service for Apache Spark.

A sample script creating a cluster with GPUs

```
#!/bin/bash
REGION=asia-east1
CLUSTER_NAME=gpu-cluster
NUM_GPUS=1
NUM_WORKERS=2
DISK_SIZE=100
GS=gs://goog-dataproc-initialization-actions-${REGION}
INIT_SCRIPT=$GS/spark-rapids/spark-rapids.sh

gcloud dataproc clusters create $CLUSTER_NAME \
    --region $REGION --image-version 3.0-debian13 \
    --public-ip-address --enable-component-gateway \
    --master-machine-type n1-standard-2 \
    --master-boot-disk-type pd-balanced \
    --master-boot-disk-size $DISK_SIZE \
    --num-workers $NUM_WORKERS \
    --worker-accelerator type=nvidia-tesla-t4,count=$NUM_GPUS \
    --worker-machine-type n1-standard-8 \
    --worker-boot-disk-type pd-balanced \
    --worker-boot-disk-size $DISK_SIZE \
    --initialization-actions $INIT_SCRIPT \
    --initialization-action-timeout 40m \
    --no-shielded-secure-boot \
    --optional-components JUPYTER \
    --metadata rapids-runtime=SPARK
```

1 Introduction to MLlib

2 Supervised machine learning with MLlib

3 Distributed algorithms for machine learning

4 Cloud computing and Google cloud platform

5 Apache Spark with Managed Service on Google Cloud Platform
- Basic usage
- Use clusters with GPUs
- Optimization

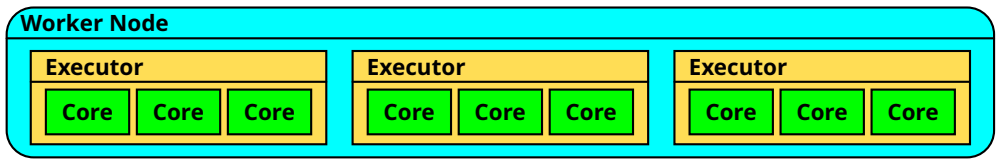
6 Summary

Optimization tips

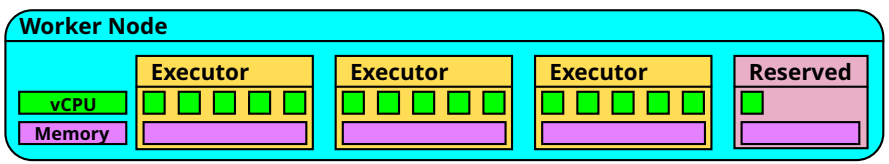
- **Use ephemeral clusters:** Create a dedicated cluster for each job, and when the job finishes, delete the cluster.

Optimization tips

- **Use ephemeral clusters:** Create a dedicated cluster for each job, and when the job finishes, delete the cluster.
- **Distribute the computational resources appropriately:**
  - Each worker node can host multiple executors.



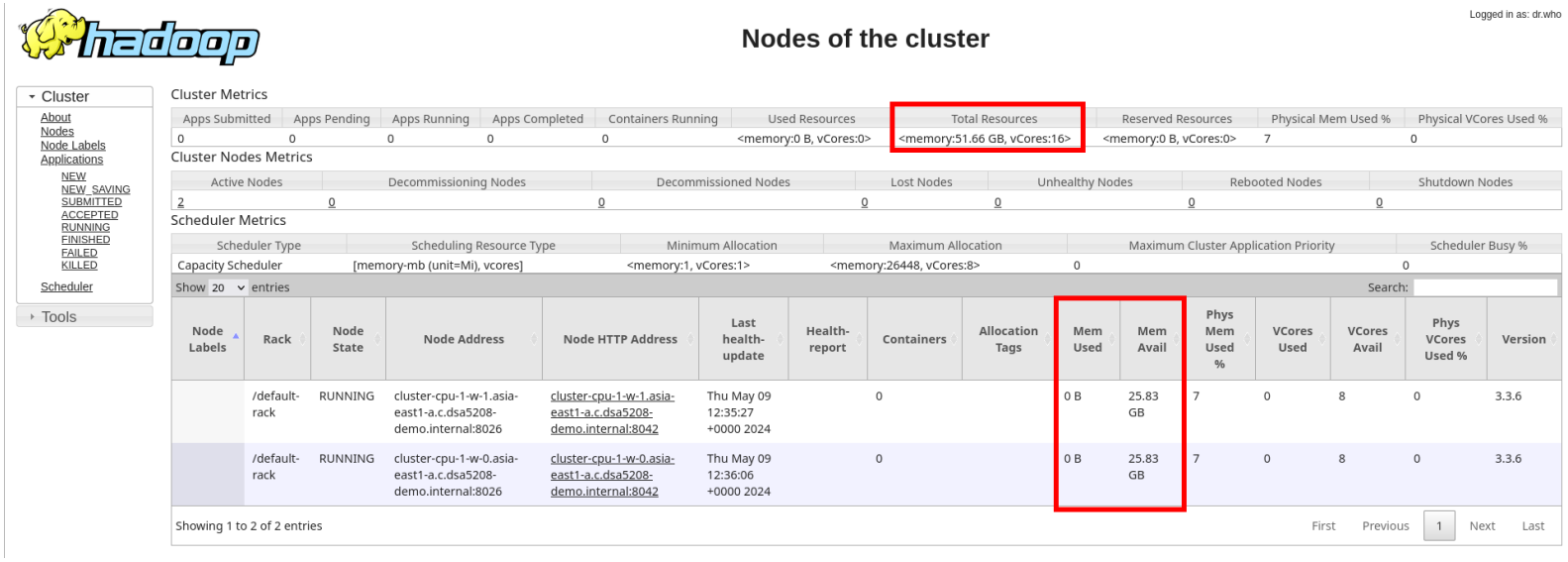
  - Too many executors per worker node will cause unnecessary communications between executors.
  - Too few executors per work node may cause bad HDFS throughput and slow garbage collection.
  - Empirically, 3–5 cores/executor provide optimal performance.
  - A general structure of resource distribution:



Example

Assume that worker nodes use the machine type `n1-standard-8`, which has 8 virtual CPUs and 30G memory.

- Each executor can have 3 cores, so that there will be two executors on each worker node, and 2 cores are reserved for the cluster manager and system resources.
- Check the amount of memory available to executors in YARN ResourceManager:



Example

- Each executor will get $25.83\text{G}/2 \approx 12.91\text{G}$ of memory.
- The total memory used by one executor includes the executor memory and the memory overhead. The memory overhead is often 10% of the executor memory, but PySpark workloads may require more.
- We can set the executor memory to be 11.7G.
- Assume we have two worker nodes. The command submitting the application is

```
$ PROPERTIES=spark.executor.instances=4
$ PROPERTIES=$PROPERTIES,spark.executor.cores=3
$ PROPERTIES=$PROPERTIES,spark.executor.memory=11700m
$ gcloud dataproc jobs submit pyspark \
    --cluster=[CLUSTER NAME] \
    --region=[REGION] [PYSPARK SCRIPT] \
    --properties=$PROPERTIES
```

Optimization tips

- **Use spot secondary workers:**
  - Secondary workers do not store data. HDFS does not run on secondary workers. They only function as processing nodes.

Optimization tips

- **Use spot secondary workers:**
  - Secondary workers do not store data. HDFS does not run on secondary workers. They only function as processing nodes.
  - Spot secondary worker are preemptible. They might be reclaimed and removed from a managed Spark cluster if they are required by Google Cloud for other tasks.

Optimization tips

- **Use spot secondary workers:**
  - Secondary workers do not store data. HDFS does not run on secondary workers. They only function as processing nodes.
  - Spot secondary worker are preemptible. They might be reclaimed and removed from a managed Spark cluster if they are required by Google Cloud for other tasks.
  - Spot VMs are available at much lower prices—up to 91% discounts for many machine types, GPUs, TPUs, and Local SSDs—compared to the on-demand price for standard VMs.

Optimization tips

- **Use spot secondary workers:**
  - Secondary workers do not store data. HDFS does not run on secondary workers. They only function as processing nodes.
  - Spot secondary worker are preemptible. They might be reclaimed and removed from a managed Spark cluster if they are required by Google Cloud for other tasks.
  - Spot VMs are available at much lower prices—up to 91% discounts for many machine types, GPUs, TPUs, and Local SSDs—compared to the on-demand price for standard VMs.
  - It is generally safe to use Spot VMs as Spark is fault tolerant.

Use preemptible or Spot workers

- To use secondary workers, we need to add a few parameters when creating a cluster:

```
$ gcloud dataproc clusters create ... \
  --num-secondary-workers [NUMBER OF SECONDARY WORKERS] \
  --secondary-worker-boot-disk-size [DISK SIZE] \
  --secondary-worker-type [WORKER TYPE]
```

  where `[WORKER TYPE]` can be one of the following:

  - `spot`
  - `non-preemptible`

  The price will be higher if the type is set to be `non-preemptible`.

- For best results, the number of preemptible workers in your cluster should be less than 50% of the total number of all workers (primary plus all secondary workers) in your cluster.

Optimization tips

- **Use local SSDs:** If an application requires high disk throughput, attaching local SSDs can improve performance. Local SSD data is ephemeral:

```
$ gcloud dataproc clusters create ... \
  --num-worker-local-ssds [NUMBER OF LOCAL SSDS PER WORKER]
```

Each local SSD has a size of 375GB for the configuration shown.

Optimization tips

- **Use local SSDs:** If an application requires high disk throughput, attaching local SSDs can improve performance. Local SSD data is ephemeral:

```
$ gcloud dataproc clusters create ... \
  --num-worker-local-ssds [NUMBER OF LOCAL SSDS PER WORKER]
```

  Each local SSD has a size of 375GB for the configuration shown.

- **Use autoscaling policies**:
  - Autoscaling policies automatically adjust the number of workers during the job execution.
  - It can be used on clusters that store data in external devices such as Google Cloud Storage. It is not intended for scaling on-cluster HDFS since the number of primary workers must be sufficient to host all HDFS data.
  - Using autoscaling policies on clusters that process many jobs can maximize the usage of resources.
  - See this link for more details about autoscaling.

1 Introduction to MLlib

2 Supervised machine learning with MLlib

3 Distributed algorithms for machine learning

4 Cloud computing and Google cloud platform

5 Apache Spark with Managed Service on Google Cloud Platform

6 Summary

Summary

Summary:

- Supervised ML with MLlib:
  - Transformers, estimators, pipelines, and hyperparameter tuning.
  - Classification and regression.
- Distributed ML Algorithms:
  - Ridge regression: matrices and vectors, `treeAggregate`.
  - Alternating Least Squares: User-item blocks and shuffle operations.
- Spark on Cloud Platforms (GCP):
  - Cluster creation and job submission.
  - Optimization strategies: Spot VMs, local SSDs, autoscaling.